Giacomelli *et al.*
EURASIP Journal on Audio, Speech, and Music Processing (2026) 2026:2
https://doi.org/10.1186/s13636-025-00436-z

EURASIP Journal on Audio, Speech, and Music Processing

SOFTWARE

Open Access

AudioSet-tools: a Python framework for taxonomy-aware AudioSet curation and reproducible audio research

Stefano Giacomelli[1*], Marco Giordano[1], Claudia Rinaldi[1,2] and Fabio Graziosi[1]

Abstract

This work presents `AudioSet-Tools`, a modular and extensible Python framework designed to streamline the creation of task-specific datasets derived from Google AudioSet. Despite its extensive coverage, AudioSet suffers from weak labeling, class imbalance, and a loosely structured taxonomy, which hinder its applicability in *machine listening* workflows. `AudioSet-Tools` addresses these issues through configurable taxonomy-consistent label filtering and class rebalancing strategies. The framework includes automated routines for data download and transformation, enabling reproducible and semantically consistent dataset generation for pre-training and downstream fine-tuning of deep learning models. While domain-agnostic, we showcase its versatility through `AudioSet-EV`, a curated subset focused on emergency vehicle siren recognition — a socially relevant and technically challenging use case that highlights structural and semantic gaps in the AudioSet taxonomy. We further provide an extensive comparative benchmark of `AudioSet-EV` against state-of-the-art emergency vehicle corpora. All source code and datasets are openly released on GitHub and Zenodo, fostering transparency and reproducibility in real-world audio signal processing research.

Keywords AudioSet, Dataset curation, Weak labeling, Machine listening, Open-source code, Audio tagging (AT), Sound event detection (SED), Python framework, Emergency vehicles (EV), Benchmarks

1 Introduction

Recent advances in deep learning (DL) and neural network (NN) architectures have significantly improved the performance of *machine listening* systems across a wide range of tasks, including general-purpose audio tagging (GP-AT), acoustic scene classification (ASC) [1–3], sound event detection (SED) [4–6], and sound source localization (SSL) [7]. These improvements critically rely on the availability of large-scale datasets that offer not only diverse acoustic contents but also semantically coherent labeling to support *supervised learning* frameworks. However, many widely used audio corpora exhibit inconsistencies that reduce model generalization and degrade transferability across tasks.

A cornerstone dataset in this field is Google AudioSet [8], a large-scale collection of over 2.1 million 10-s audio segments sourced from YouTube videos and *weakly annotated*, meaning that the presence of a sound event is indicated for the entire clip without temporal alignment, i.e., as tags. Labels follow a hierarchical taxonomy of 527 classes, constructed by extracting over 3000 candidate sound terms from web-scale text corpora using Hearst patterns [9], and subsequently curated and mapped to Freebase Knowledge Graph identifiers [10]. This relational structure supports multi-label classification and

*Correspondence:
Stefano Giacomelli
stefano.giacomelli@graduate.univaq.it
[1] Department of Information Engineering, Computer Science and Mathematics (DISIM), University of L'Aquila (UnivAQ), Via Vetoio, L'Aquila 67100, Abruzzo, Italy
[2] National Inter-University Consortium for Telecommunications (CNIT), University of L'Aquila (UnivAQ), Via Vetoio, L'Aquila 67100, Abruzzo, Italy

semantic abstraction, making AudioSet a foundational resource for large-scale machine listening applications.

Despite its value, the `Standard` AudioSet release presents limitations that hinder its applicability in downstream DL workflows. Labels lack time alignment [11]; class imbalance is severe — especially for rare or context-specific events such as emergency vehicle (EV) sirens [12–14] - and its taxonomy is loosely enforced, resulting in semantic overlap and redundancy. For example, `Siren`, `Police car (siren)`, and `Ambulance (siren)` often appear as independent categories with no explicit hierarchical linkage. This can lead to clips annotated with specific subcategories that omit their parent class (e.g., `Siren` or `Emergency vehicle`), thus undermining consistency during model training and inference.

The 2021 AudioSet-`Strong` extension [15] partially addresses these issues by providing time-aligned (*strong*) labels and human-validated annotations for a limited subset. However, this improved annotation quality comes at the cost of reduced dataset scale, reducing coverage and increasing imbalance for rare events.

AudioSet is distributed under a Creative Commons Attribution 4.0 International license (CC BY 4.0) and is provided as a collection of `.csv` metadata files. Each file lists 10-s audio segments with one or more class labels specified as MID identifiers. These identifiers must be resolved into Human-Readable Format (HRF) (e.g., MID: `/m/03j1ly` → HRF: `Emergency vehicle`) exploiting a dedicated ontology file (`class_labels_indices.csv`), which is separately released under the ShareAlike (CC BY-SA 4.0) license. The dataset is partitioned into three disjoint subsets:

- *Balanced training set*: 22 176 segments with at least 59 samples per class.
- *Evaluation set*: 20 383 segments with balanced class distributions.
- *Unbalanced training set*: over 2 million segments with long-tail class distribution.

Although structurally defined, AudioSet lacks an official toolchain for audio retrieval, label filtering, or metadata querying. Researchers must rely on manual `.csv` parsing and custom scripts for pre-processing metadata and downloading audio data, which risks inconsistent handling and undermines the reproducibility of experiments.

To fill this gap, several open-source pre-processing tools have emerged, including `audioset_utils` [16], `audioset-processing` [17], and `Fast-Audioset-Download` [18]. These tools typically support basic metadata parsing, segment filtering, and audio downloading (often using `yt-dlp` [19] and `ffmpeg` [20]). However, they remain fragmented, lack taxonomy-consistent processing, and offer limited support for composability, deterministic behavior, or scalable integration in larger machine listening pipelines. Unlike these hands-on initiatives, our `AudioSet-Tools` unifies hierarchical label processing, taxonomy-consistent dataset construction, and reproducible pre-processing within a modular and easily maintainable framework.

The landscape of environmental sound datasets has also evolved toward taxonomic structuring. Datasets such as *UrbanSound8K* [21] and *FSD50K* [22] embed hierarchical labeling principles inspired by AudioSet. This shift has enabled improved transfer learning strategies, particularly in foundational model training and cross-dataset ontology mapping. Resources like the Standardized Audio events Label Taxonomy (SALT) [23] formalize this integration by aligning 24+ audio tagging datasets under a shared semantic structure. In this perspective, effective filtering and restructuring of the AudioSet taxonomy could provide a systematic solution to analogous issues in other datasets connected through SALT. Recent studies have also proved how taxonomic consistency improves models training generalization and facilitates multi-task reasoning [24, 25], for example by enabling a NN trained on `Siren` to extend its knowledge to related labels such as `Alarm` or `Horn`.

These challenges and opportunities collectively motivate the need for a robust and extensible toolkit for filtering, analyzing, and restructuring AudioSet. Operating directly on the full release, while preserving taxonomic consistency, enables scalable construction of task-specific datasets, fostering stronger embeddings and hierarchical generalization for downstream real-world applications.

In this paper, we introduce `AudioSet-Tools`, a modular Python framework designed for reproducible and task-oriented dataset construction from AudioSet. It addresses all previous technical and conceptual limitations by enabling structured label filtering, taxonomy configuration [23], class balancing, and robust automatic segments download and pre-processing. Our framework supports both the `Standard` and `Strong` AudioSet releases and is distributed as a ready-to-use GitHub repository that can be cloned as a workspace. This design avoids the need for traditional Python's `pip`-based installation and dependency maintenance, while still allowing users to directly inspect and adapt its core modules for ad-hoc applications. In addition, the system enables flexible metadata parsing and taxonomy configuration, meaning that target sub-taxonomies for specific tasks can be defined without modifying the original metadata or ontology files provided with the official AudioSet releases. `pySALT` [26] usage can be optionally integrated

to streamline label validation and enhance interoperability with other supported ontologies.

To demonstrate the framework's impact, we present `AudioSet-EV`, a curated dataset for EV siren recognition — a task relevant to autonomous driving, smart city systems, and real-time urban monitoring [12–14]. Related EV-siren corpora, such as *sireNNet* [27], *LSSiren* [28, 29], *ESC-50* [30], *UrbanSound8K* [21], and *FSD50K* [22], exhibit limited class coverage, contextual diversity, and taxonomic consistency. Using our framework, we build `AudioSet-EV` via ontology-consistent label filtering and the integration of urban distractor classes (e.g., car horns, civil or military alarms, and other periodic signals that often challenge automatic machine listening models; Sect. 3). We then apply class balancing and waveform-level standardization (uniform resampling to 32 kHz, 24-bit Pulse Code Modulation encoding) to ensure consistency across all audio files (Sect. 3.1.3). The resulting dataset includes 16 910 annotated clips (≈47 h), organized into a binarized selection (EV sirens vs. non-EV urban distractors) spanning multiple AudioSet branches e.g., vehicles, alarms, horns, and traffic sounds.

This work overall provides three main contributions: (1) the release of `AudioSet-Tools`, a documented framework for taxonomy-consistent dataset curation and robust download automation; (2) the construction of `AudioSet-EV`, a reusable benchmark for siren classification and detection[1]; and (3) the open release of all code, configurations, and datasets via GitHub and Zenodo, supporting research reproducibility. We also evaluate the resulting dataset using a baseline NN model to assess classification performance and consistency during training as well as its knowledge transferability across benchmark corpora [31].

The remainder of this paper is structured as follows: Sect. 2 details the architecture and implementation of the `AudioSet-Tools` framework, outlining each processing module and its role in dataset curation. Section 3 describes the `AudioSet-EV` case study, including rationale, selection criteria, and structural statistics. Section 4 presents a comprehensive benchmarking of `AudioSet-EV` against existing *state-of-the-art* EV corpora, evaluating dataset scale, taxonomic coherence, and recognition performance. Finally, Sect. 5 discusses limitations, key findings, and future directions for extending the application of the framework to other audio domains.

[1] While the core release of `AudioSet-EV` includes only binary weak labels for classification, EV detection tasks are supported through the integration of `AudioSet-Strong` metadata. Specifically, all samples appearing in both the Standard and Strong releases of AudioSet are provided with their corresponding time-aligned annotations, enabling SED experiments.

2 Methodology — implementation: AudioSet-tools

This section introduces the design principles and implementation strategies of `AudioSet-Tools`. Several open-source projects have aimed to simplify AudioSet pre-processing workflows, focusing on metadata parsing, label filtering, and audio downloading. While these tools provide valuable insights — particularly in automating content retrieval and partial segment extraction — they remain task-fragmented, lack taxonomy-consistent processing, and offer limited long-term support or extensibility. A comparative overview of these tools is summarized in Table 1, which outlines their main functionalities, strengths, and current limitations as evaluated in our analysis.

Collectively, these tools demonstrate useful strategies such as the pre-slicing of audio segments via `ffmpeg` [20], an open-source suite for multimedia format processing, and automated retrieval of YouTube content using `yt-dlp` [19], a command-line interface downloader (with a Python API) supporting Dynamic Adaptive Streaming over HTTP (DASH) - HTTP Live Streaming (HLS) manifest parsing and high-quality stream extraction. `AudioSet-Tools` inherits and consolidates these strategies within a unified, modular, and robust framework. Designed with accessibility in mind, the framework includes comprehensive documentation, ensures deterministic behavior, and facilitates seamless integration into machine listening workflows.

At a high level, the framework is structured into modular components corresponding to the main stages of audio dataset preparation: label *filtering*, class *balancing*, audio *downloading*, and waveform *pre-processing*. It is compatible with both the `Standard` and `Strong` releases of AudioSet, natively supporting their respective metadata formats — `.csv` for `Standard`, and tab-separated values (`.tsv`) for `Strong`. This distinction has practical implications for metadata parsing and processing: in the `Standard` format, each row corresponds to a 10-s audio segment and may include multiple labels for co-occurring events; in contrast, the `Strong` format provides time-aligned annotations with each row tracking the presence of a specific class within a precise time window, resulting in multiple entries per original YouTube ID, each corresponding to a distinct temporally annotated event.

To accommodate these structural differences, each processing function in our framework is implemented in two variants — identically named (e.g., `select_by_label`) but organized into separate modules: `filters` and `filters_strong` for label-level operations, and `utils` and `utils_strong` for support utilities (Fig. 1). This modular duplication ensures format-specific logic while

Table 1 Comparison of existing AudioSet pre-processing toolkits. Tools are evaluated along technical and metadata/structural dimensions

Tool	Technical aspects	Metadata/structural aspects
`Audioset_utils` [16]	HuggingFace-compatible loaders	Non-hierarchical label filtering
	Download routines available	Low repository activity (maintainability issues)
`Audioset-processing` [17]	CLI with single-threaded downloads	Minimal support for taxonomy-consistent processing
	Basic DSP routines `pyffmpeg` compatibility issues	
	Limited error handling	
`Fast-Audioset-Download` [18]	Segment-level downloading with `ffmpeg/yt-dlp`	Outdated dependencies
	JSON result reporting	No taxonomy-consistent label/sample processing
`AudioSet Download` [32]	Python 3.9-only compatibility	No long-term support
	Parallel downloading supported (minor bugs reported)	No label selection
	Unstable download routines	
`AudioSet-Tools`**(proposed)**	**Robust, automated downloading**	**Taxonomy-consistent filtering and validation**
	Structured error & result logging	**Class rebalancing and reproducible metadata restructuring**
	Cookie validation & refresh	**Compatibility with `pySALT`Compatibility withfor cross-dataset ontology mapping**
	Cross-platform support and reproducible pipelines	**Metadata support for the AudioSet-Strong release**

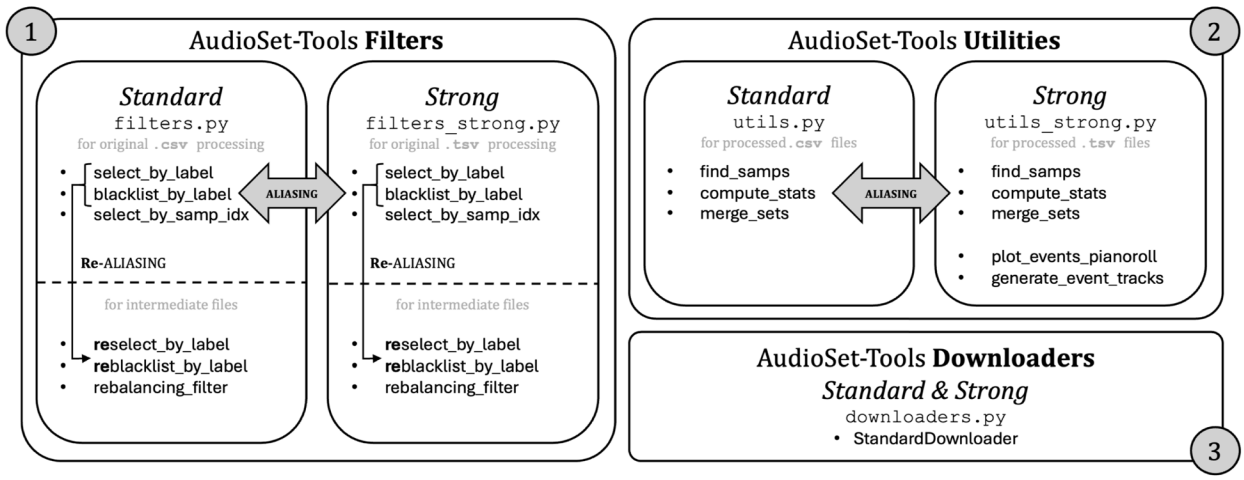

Fig. 1 Architecture and data flow of the *AudioSet-Tools* framework. Each component corresponds to a stage in the dataset processing pipeline, supporting reproducible filtering, label processing, downloading, and audio pre-processing

maintaining a consistent user interface, enabling seamless transitions between AudioSet releases within the same project. The framework functionalities are grouped into two main clusters:

- *Filters*, which provide functions for selecting or excluding samples (e.g., blacklisting) based on semantic labels or sample indices;
- *Utilities*, which offer support routines for merging intermediate metadata sets, analyzing label distributions, inspecting dataset contents, and generating visual and textual reports.

The `Standard` and `Strong` releases share the same underlying YouTube segments, with the latter providing temporally refined annotations. `AudioSet-Tools` leverages this overlap through a unified `StandardDownloader` class that handles both metadata formats without code duplication or user intervention. This design ensures maintainability, efficiency,

Table 2 AudioSet-Tools filters naming convention: *labels_file* is the AudioSet MID-to-HRF map, *data_file* and *in_file* are the input.csv/.tsv files to be processed, and *out_file* is the intermediate/output.csv/.tsv generated

Function name	Arguments	Output	Strong support
`Select_by_label`	`labels_file` (file path) `data_file` (file path) `target_labels` (str, list[str]) `out_file` (filename & path) `verbose` (boolean)	`.csv/.tsv` file with target label(s) occurrences only	✓
`Blacklist_by_label`	`labels_file` (file path) `data_file` (file path) `target_labels` (str, list[str]) `out_file` (filename & path) `verbose` (boolean)	`.csv/.tsv` file with all target label(s) occurrences excluded	✓
`Select_by_samp_idx`	`data_file` (file path) `start_idx` (int) `end_idx` (int) `out_file` (filename & path) `verbose` (boolean)	Subset `.csv/.tsv` file by sample indexing	None
`Rebalancing_filter`	`in_file` (file path) `labels_file` (file path) `focus_labels` (str, list[str]) `out_file` (filename & path) `verbose` (boolean)	Balanced `.csv/.tsv` file with target uniform class distributions	✓

The parameter `in_file` is used only in the `rebalancing_filter` function (standard and strong) and corresponds to `input_csv/input_tsv` in the Python source code. This altered naming convention was adopted to improve readability in experimental contexts, where the rebalancing filter may be applied independently of other functions. In all other cases, the more general naming `data_file` is used for syntax consistency

and a consistent pipeline for audio retrieval and pre-processing.

All modules share a unified interface, supporting verbose console logging, stateless execution, and arbitrary chaining. Importantly, `AudioSet-Tools` is distributed as an open framework: not as a monolithic Python package but as a fully inspectable and customizable codebase. This encourages users to tailor each component to domain-specific use cases, while preserving transparency and reproducibility.

The implementation leverages a state-of-the-art Python ecosystem: `pandas` and `numpy` [33] for data handling; `yt-dlp` and `ffmpeg` for content retrieval and slicing; `resampy` [34], `soundfile`, and `torchaudio` [35, 36] for audio re-sampling and dataset structuring; and `matplotlib` for visualizations. Additional dependencies such as `tqdm`, `certifi`, `brotli`, `requests`, `curl_cffi`, and `SecretStorage` ensure compatibility with modern web protocols and secure access to restricted content. A summary of the framework architecture is provided in Fig. 1.

2.1 Label filtering and taxonomy-consistent curation

As a taxonomy-driven framework, `AudioSet-Tools` provides a flexible suite of filtering utilities that enable fine-grained and reproducible selection of task-specific subsets from the AudioSet corpus. This filtering stage is crucial for constructing semantically controlled datasets tailored to specific machine listening tasks, such as binary event detection or hierarchical multi-class tagging[2]. The utilities operate seamlessly on both the `Standard` and `Strong` releases, adapting their internal logic to the specific metadata structures and levels of annotation granularity.

Each filtering function also has a `re`-prefixed variant (e.g., `reselect_by_label`), designed for metadata where MIDs have already been resolved to HRF labels. The prefix indicates *HRF-re*solved inputs and adds additional consistency checks on structure, headers, and column alignment. This mechanism mitigates issues such as separator encoding or header mismatches, ensuring robustness when chaining multiple pre-processing steps.

The available filtering functions are summarized in Table 2, and include:

[2] In this work, a *label* denotes an atomic category as defined in the AudioSet ontology (e.g., `'Ambulance (siren)'`), whereas a *class* may correspond either to such a single label or to a higher-level grouping of labels (e.g., *Vehicle-related sounds*) introduced for analysis or balancing purposes.

Select_by_label() isolates all metadata entries that contain one or more of a user-specified set of target labels. The function first resolves HRF labels to AudioSet MIDs via the provided taxonomy, then identifies all matching entries. In the `Standard` release, this entails selecting metadata rows corresponding to clips annotated with at least one relevant class; in the `Strong` version, all time-aligned rows associated with those labels are retained. This operation is suitable for constructing semantically precise True Positive (TP) subsets for supervised learning tasks — for example, isolating clips containing vehicle sirens from a large pool of general urban recordings. A study illustrating this use case is further discussed in Sect. 3.1.1.

Technical details: the function loads the provided labels `display_name→mid` map file, builds the target MID Python set (unique assignments), and filters samples (rows) by *any* MID match in the label column: in the `Standard` case, MIDs are normalized from columns 4+ (comma-split text); in the `re-processed.csv` case, column 4 already stores a serialized list, which is safely parsed using Python's `ast.literal_eval`. The original metadata header is preserved, and selected rows are rewritten in the intermediate file, as `row[:3] + [list_of_mids]`.

Blacklist_by_label() performs the complementary operation, excluding all entries that match *any* of the specified labels. In the `Standard` case, entire metadata rows are discarded if any undesired label is found; in the `Strong` format, only the rows containing matching annotations are removed. This function is particularly useful for constructing reliable True Negative (TN) classes or filtering out semantically confounding events from an otherwise heterogeneous dataset (Sect. 3.1.2).

Technical details: analogous to `select_by_label`, labels are first resolved to target MIDs and then rows are *excluded* if they contain *any* MID in the input exclusion list. Label normalization follows the same schema (raw `.csv` vs. re-processed `.csv`).

Select_by_samp_idx() enables slicing or subsetting of the dataset based on sample indexing. It does not involve any label-based logic and is applicable only to the `Standard` format, where sample indices directly correspond to metadata rows. This function supports controlled experimentation, deterministic validation of data-splits, or systematic resampling strategies.

Technical details: the input slice is half-open `[start_idx, end_idx)` (inclusive at the start, exclusive at the end). The metadata header is always preserved. This enables deterministic, reproducible slicing of `.csv/.tsv` files *without* any label-dependent logic.

Rebalancing_filter() addresses the pervasive issue of class imbalance in AudioSet. It computes occurrence statistics of the target labels and uniformly downsamples all overrepresented classes to match the least frequent one. In the `Standard` release (`.csv`), rebalancing operates at the clip level: a single sample may contain multiple target labels and is therefore counted in the statistics of each of the corresponding classes. The entire audio clip is always preserved, even when associated with multiple labels.

In contrast, the `Strong` release (`.tsv`) is already structured at the event level, with each row referring to a single time-aligned annotation. In this case, rebalancing is performed directly on these individual annotations, resulting in a uniformly balanced set of event-level instances, without altering the underlying audio files.

Technical details: the function equalizes per-label frequencies over a focus set F (expressed in `display_name` and resolved to MIDs). For each $\ell \in F$, the count c_ℓ is computed on current rows (*clip-level* multi-label in `Standard`, *event-level* in `Strong`), and $m = \min_{\ell \in F} c_\ell$ is set as the per-label target. For every label, we then sample *without replacement* m elements among those containing it; the output is the set union of all selected elements. In the `Standard` case a multi-label row may contribute to multiple counts but is written only once to the final file; in the `Strong` case rebalancing acts on *event rows* and does not alter the audio files. The associated random generator is seeded for reproducibility. *Note:* "`re*`" variants add schema and field checks for pre-decoded files.

AudioSet's taxonomy lacks a strict hierarchy, meaning that semantically related classes — such as `Siren`, `Police car (siren)`, and `Ambulance (siren)` — remain structurally independent. Without user-defined mappings or external ontologies such as SALT [23], filtering one class may unintentionally exclude others. `AudioSet-Tools` is intentionally agnostic in this respect, delegating taxonomic alignment and awareness to the user.

2.2 Metadata utilities

In addition to filtering routines, `AudioSet-Tools` provides a set of auxiliary utilities supporting common metadata processing tasks across different curation stages. These are implemented in two parallel modules — `utils.py` and `utils_strong.py` — tailored to the structural properties of the `Standard` and `Strong`

AudioSet releases, respectively. Despite format differences, both modules adopt consistent naming conventions and function interfaces. Provided functionalities can be grouped into three main categories:

2.2.1 Metadata merging and de-duplication

When handling multiple intermediate metadata files (e.g., from class-specific filtering), merging them into a single coherent dataset is often necessary. The function `merge_csv_files()` concatenates such files while optionally applying de-duplication logic based on YouTube segment identifiers. For the `Standard` release, this involves removing redundant entries with identical segment IDs; for the `Strong` version, uniqueness is enforced at the annotation level by also considering timestamps. This ensures that downstream filtering and balancing operate on consistent, non-overlapping data.

All filtering and indexing operations in `AudioSet-Tools` are designed to produce explicit `.csv/.tsv` outputs at each step. This promotes transparency and modularity, allowing users to check intermediate results and debug pipelines effectively. To reduce storage overhead, users can remove these *temporary* files once the pipeline correctness is verified (e.g., via Python `os.remove`[3]).

2.2.2 Content inspection and descriptive statistics

Understanding label distribution is crucial for class balancing and semantic pruning. The functions `count_samples_by_label()` (`Standard`) and `count_samples_by_event()` (`Strong`) compute per-class occurrence statistics and return results as Python dictionaries or console outputs. Another shared utility, `find_samps_by_samps()`, identifies common entries between two metadata files by comparing segment IDs (with or without time alignment), facilitating tracking across parallel pipeline branches.

2.2.3 Dataset reporting and visualization

To enhance interpretability and reproducibility, the `compute_stats()` function generates textual (`.json`) reports summarizing dataset size, label coverage, and optional label-wise statistics. The `plot_distribution()` function complements this with visual bar plots of class frequency, supporting diagnostic comparisons before and after filtering or rebalancing.

For fine-grained event analysis in the `Strong` release, two additional tools are provided. `plot_events_pianoroll()` visualizes temporal annotations of multiple events on a MIDI-style *piano roll*, helping users assess co-occurrence and overlap within a segment. `generate_event_tracks()` converts annotations into fixed-resolution binary tracks for training detection models. Each output item is a `tuple`, pairing a segment + label identifier with a binarized `numPy.ndarray` or `torch.Tensor`, where `1s` indicate presence and `0s` absence of the target class over time bins of user-defined size (`bin_size`, in seconds), as illustrated in Fig. 2.

2.3 Downloading and pre-processing automation

The `StandardDownloader` class encapsulates an end-to-end pipeline for retrieving, processing, and organizing audio content from the AudioSet corpus (Fig. 3). Implemented as a Python `context manager`, it provides a robust and configurable interface for YouTube-based segment extraction, supporting a wide range of dataset curation workflows. The pipeline is structured into four sequential stages:

2.3.1 Stage 1: initialization and configuration

The downloader is initialized with an AudioSet metadata file and its corresponding MID-to-HRF mapping. Optional parameters include the resampling rate (`target_sr`), audio channel mode (`channels_proc`, supporting `stereo`, `mono_split`, and `mono_red`), an amplitude normalization flag, and an optional cookie file for authenticated content. An output directory is created, named after the metadata file, with consistent filename formatting for downloaded samples. Internal attributes track download status, label coverage, and operational parameters.

2.3.2 Stage 2: context-based metadata loading

Used within a Python `with` block, the downloader class ensures automatic resource management. Upon entry, metadata and label mapping files are loaded and validated. If the metadata lacks a `downloaded` column (used to track progress), it is appended dynamically with `False` flags. This enables resumable and incremental downloads across sessions.

This mechanism also exposes a significant limitation of the AudioSet corpus: due to its dependency on YouTube as a hosting platform, many referenced segments are no longer accessible. The underlying causes are multifold: voluntary deletions by content owners, copyright enforcement actions, platform policy changes, and obfuscation via access restrictions or password protection. Our empirical analysis (Sect. 4.1.1) confirms that as of May 2025, a considerable fraction of the content is unavailable due to deletions, copyright enforcement, or access restrictions. This condition, not explicitly addressed in prior literature [8, 15], undermines the long-term

[3] Future releases will extend this mechanism by adding an optional boolean parameter (e.g., `del_inter_file: bool = True/False`) to functions that generate output files, enabling automatic cleanup of intermediate results when desired.

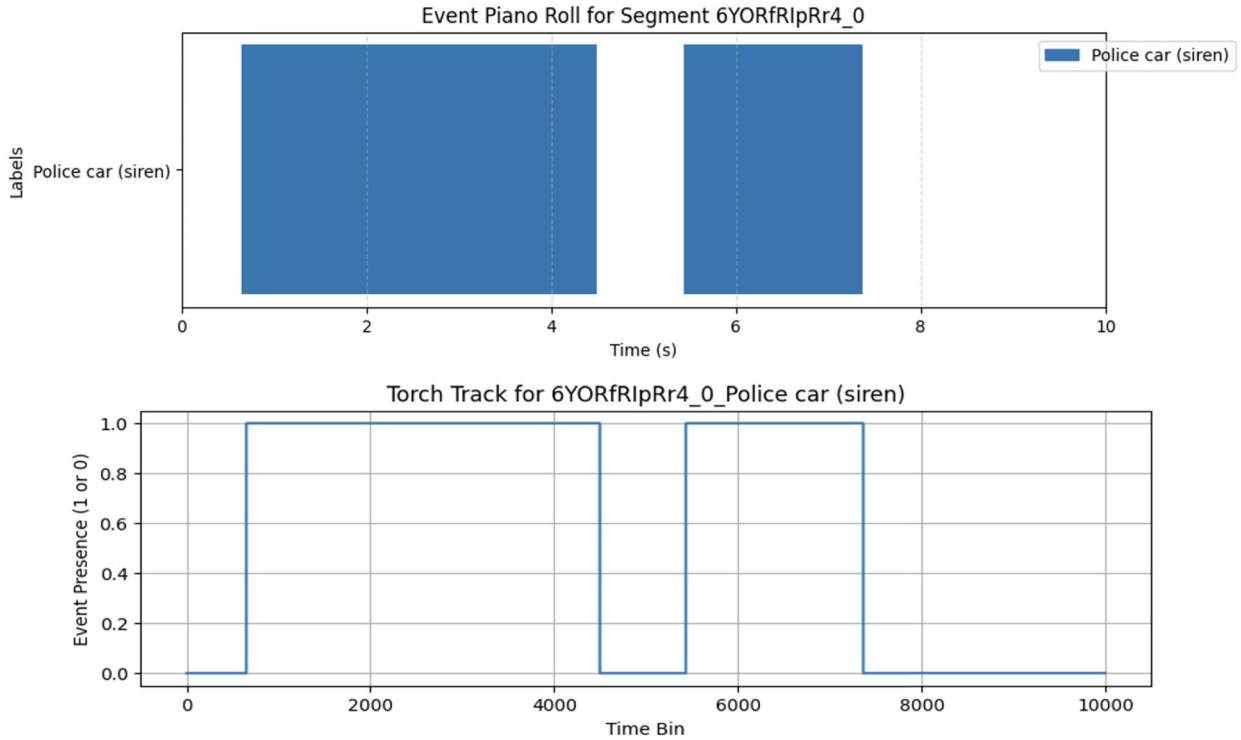

Fig. 2 Example of a MIDI *piano-roll* plot (upper) and a *One-Hot* encoded AudioSet *Strong* PyTorch track, for a specific dataset sample

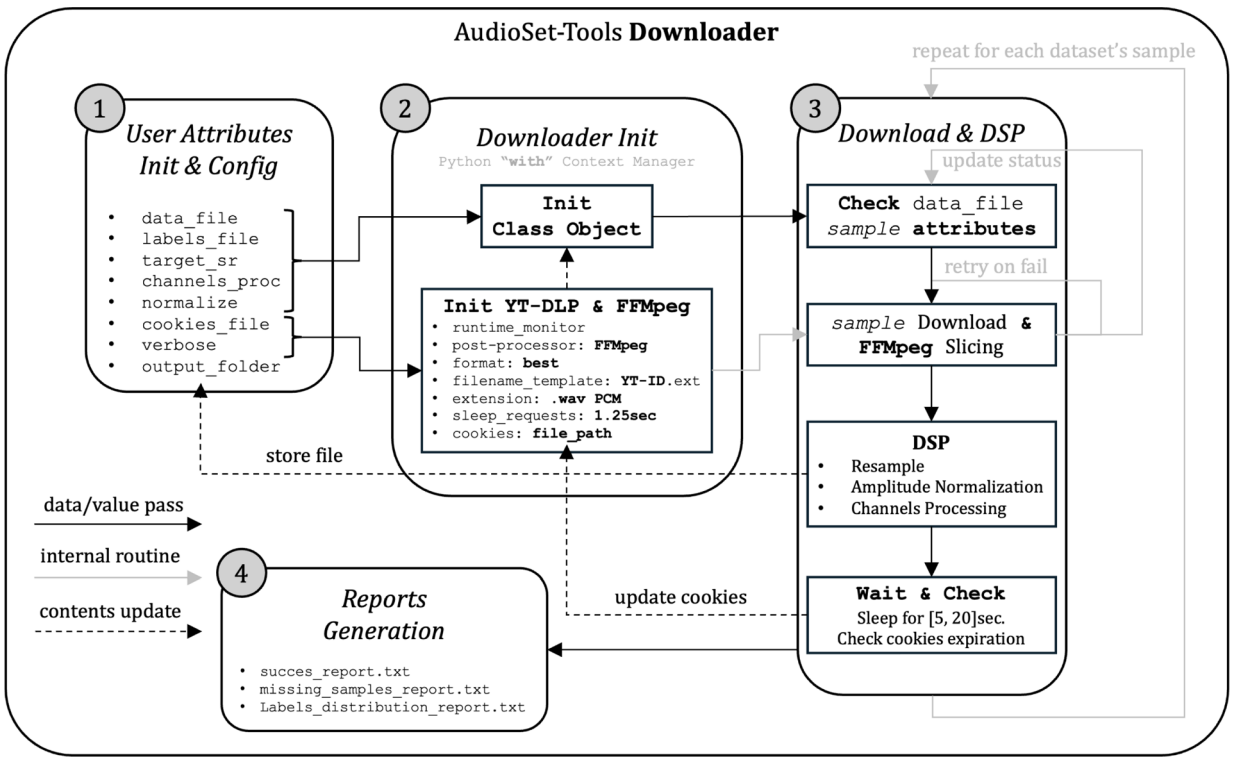

Fig. 3 Internal architecture of the *StandardDownloader* class

consistency of AudioSet subsets such as `balanced_train` and `eval`.

2.3.3 Stage 3: audio download and DSP pre-processing

Each entry is processed by the **download_and_process** method, leveraging the `yt-dlp` API to fetch the highest-quality audio stream for the given YouTube ID. Audio is saved as uncompressed `.wav` files after being clipped to the desired segment duration via `ffmpeg`.

Although exported in PCM format, content quality reflects the original upload source — sometimes compressed, edited, or degraded — which is preserved without modification. A stable intermediate naming format (`outtmpl: %{id}.%{ext}`) facilitates subsequent DSP routines. A fixed delay (1.25s) is imposed between requests, and optional cookie-based authentication is supported via the `cookiefile` parameter.

During the downloading loop, the framework inspects potential exceptions raised by `yt-dlp` to handle common failure scenarios. In particular, it detects messages associated with access restrictions often described as *YouTube "shadow bans"* — a form of content unavailability triggered by geographic, authentication, or anti-bot restrictions. When such messages are encountered, the system halts the entire download sequence to prevent unnecessary requests. If the exception instead suggests an expired or invalid authentication session (e.g., `Sign in to confirm you're not a bot`), an automated recovery routine is triggered via the **refresh_cookies** method. This method launches a default browser session, prompting the user to manually authenticate the YouTube account and regenerate a valid `cookies.txt`[4] (to be saved inside the current working directory). The downloader continuously monitors the session and, upon closing the browser, updates the cookie file into the `yt-dlp` configuration. This mechanism ensures robustness and continuity when accessing restricted content, without requiring a full restart of the pipeline. The downloaded sample DSP pipeline includes:

1. *Re-sampling*: performed through optimized band-limited interpolation with `resampy` [34, 37], if the input sample rate differs from the target one.
2. *Amplitude Normalization*: optional peak-normalization, scaling the signal to a maximum absolute value of 1.0.
3. *Channel Processing*: user-defined mode determines whether stereo content is preserved, channel-based split, or down-mixed to mono.

[4] We highly suggest the use of Mozilla Firefox browser on Linux, combined with the `cookies.txt` add-on.

Processed files are saved with informative suffixes (e.g., `_mono.wav`, `_left.wav`) in the designated output folder. To reduce the likelihood of IP bans due to high request rates, randomized pauses between 5 and 20 s are inserted between download retries.

2.3.4 Stage 4: structured reporting

On context exit, the downloader generates a set of diagnostic files:

- `success_report.txt` — list of successfully retrieved files and labels;
- `missing_samples_report.txt` — list of failed downloads with corresponding YouTube IDs;
- `labels_distribution_report.txt` — summary of class frequencies within the retrieved dataset.

These logs ensure reproducibility, traceability, and support detailed inspection of the download outcomes. Thanks to its modular architecture and metadata compatibility with both AudioSet formats, the `Standard-Downloader` class integrates smoothly into broader pipelines and remains suitable for both custom and large-scale dataset generation workflows. Beyond reproducibility and transparency, automated handling of download failures, session recovery, and traffic throttling represents a key novelty of our framework. Unlike previous toolkits, `AudioSet-Tools` provides a robust and self-contained mechanism for retrieving large volumes of audio segments under real-world network and platform constraints, thus significantly reducing failure rates and simplifying dataset construction.

3 Application case study: AudioSet-EV

We demonstrate the modularity and usability of `AudioSet-Tools` through a real-world use case: the construction and validation of `AudioSet-EV`, a curated subset of Google AudioSet specifically tailored for emergency vehicle (EV) siren recognition.

EV siren detection plays a key role in smart city infrastructures and autonomous mobility, with implications for road safety, urban coordination, and assistive technologies [12–14]. Timely identification of *moving sirens* supports anticipatory behaviors in both humans and autonomous systems, contributing to faster reactions and informed decisions. Technological applications span from Advanced Driver Assistance Systems (ADAS) and Autonomous Driving Systems (ADS) to infrastructure-integrated alerting modules [38, 39]. From an acoustic perspective, EV sirens are intentionally designed to ensure perceptibility within cluttered sonic environments. Typical implementations, widely adopted across

different regions, include the *wail* (slow sinusoidal modulation from approximately 400 to 1600 Hz over 4–6 s), the *yelp* (rapid cyclic modulations between roughly 1000 and 2500 Hz, with a period of about 0.5 s), and the *hi-lo* (alternating tones near 650 and 900 Hz every 1 s), each characterized by distinctive time–frequency structures [40, 41]. These frequency ranges and profile periods are not universally fixed but can vary depending on local regulations, manufacturer design, and operational requirements. Additional variants such as the *piercer* (sharp alternations peaking > 2 kHz), the *rumbler* (low-frequency cycles around 180–400 Hz), and the broadband *air horn* further expand the expressive palette, adapting to diverse environmental demands [42, 43]. The automatic sensing of these signals are further complicated by urban reverberation, *Doppler* shifts, occlusions, and overlapping noise sources — posing significant challenges to detection systems.

Despite its operational relevance, training resources for EV siren detection remain sparse and fragmented. We surveyed available EV corpora and found only two open audio datasets specifically focused on this domain: *sireNNet* [27], comprising roughly 400 3-s clips per class (`firetruck`, `ambulance`, `police`, `traffic`), including both ambient and manually isolated clean siren clips; and *LSSiren* [28, 29], a larger dataset of 1800 recordings (3–15 s), balanced between the `Ambulance` and `Road Noise` classes. The former offers clean and well-labeled training inputs but lacks source diversity; the latter improves capture variability (e.g., microphone types and distances), but provides only coarse taxonomic granularity with binary labels.

While our experiments remain focused on the general-case scenario of binary classification (*siren* vs. *non-siren*), the availability of sub-class labels would broaden the range of possible applications. Such granularity would enable, for example, urban traffic management systems that prioritize different emergency vehicles based on size and urgency, multi-modal tracking in combination with video feeds, healthcare monitoring through statistics on ambulance response times, forensic or incident analysis, and simulation and training for autonomous vehicles. From this perspective, the absence of sub-class specifications in current corpora represents an inner limitation for more specialized use cases.

Additional General-Purpose (GP) environmental sound datasets partially cover EV sirens. *ESC-50* [30] contains 2000 mono clips across 50 classes, yet includes only 40 examples labeled as `siren`, with no subclass or co-occurrence annotations. *UrbanSound8K* [21] offers 8732 urban samples grouped into 10 classes (including `siren`), split into 10 folds for evaluation, but lacks hierarchical structuring and detailed metadata. *FSD50K* [22], the largest corpus considered, features over 51,000 multi-labeled clips annotated via Freesound crowdsourcing and aligned to the AudioSet ontology. However, siren-related tags (e.g., `siren`, `ambulance`, `police car`) are sparsely represented, inconsistently mapped, and lack any enforced taxonomic structure. While these corpora offer complementary coverage, they share several structural limitations: (1) low specificity for EV-related labels, (2) limited taxonomic expressiveness and label disambiguation, (3) inconsistent metadata formats and annotation standards, and hence (4) poor interoperability for benchmarking, since each corpus requires different pre-processing pipelines (e.g., varied audio formats, clip durations, metadata structures, and annotation styles). This lack of standardized workflows restricts reproducibility, generalization, and the ability to conduct cross-dataset evaluations — ultimately hindering robust model development and fair cross-corpus benchmarking.

A wide variety of DL architectures have been proposed to address siren recognition, including Convolutional Recurrent Neural Networks (CRNNs) based on bi-directional Gated Recurrent Units (GRUs) [39], *You Only Look Once* (YOLO)v5-based spectrogram detectors trained on synthetic in-vehicle data [39], lightweight CNNs for real-time deployment [44], and hybrid backbones combining EfficientNet, 1D-CNNs, and Self-Attention mechanisms [45]. Other studies explored *ensemble voting* schemes across Multilayer Perceptron (MLP), CNN and RNN architectures [46], *few-shot* learning strategies on *prototypical networks* [47], and source-aware localization NN [48]. Nevertheless, many of these rely on custom or synthetic datasets, lack transparent sample provenance, and rarely publish their datasets metadata, making replication and comparative evaluation difficult. A more detailed and systematic overview of existing architectures and methodologies for siren recognition is provided in our related work [31].

To fill this gap, we introduce `AudioSet-EV`, a semantically coherent and reproducible subset of AudioSet. It is constructed using label-aware filtering (`Emergency vehicle`, `Siren`, `Ambulance (siren)`, `Police car (siren)`, `Fire engine, fire truck (siren)`), blacklist rules for ambiguous categories, and taxonomy-consistent generalization strategies. Each clip is linked to metadata, taxonomic context, and provenance (YouTube ID, timestamp), traceable via structured `.csv`/`.tsv` files generated by `AudioSet-Tools` and distributed via Zenodo. In contrast to existing corpora, `AudioSet-EV` offers: (1) semantic consistency across labels[5], (2) transparent metadata and processing

[5] Here, semantic consistency is explicitly intended as inter-dataset consistency in label definitions and taxonomic structure.

traces, and (3) a benchmarking-ready structure. This supports model evaluation in realistic scenarios and enables research on robustness, transferability, and generalization in acoustic event recognition.

3.1 Contents curation process

The `AudioSet-EV` subset was built through a systematic and reproducible procedure based on the configuration-driven pipeline provided by `AudioSet-Tools` (Sect. 2). The curation process began with a taxonomy-aware definition of the target label space, isolating a small set of semantically precise *Positive* samples: `'Police car (siren)'`, `'Ambulance (siren)'`, `'Fire engine, fire truck (siren)'` (*leaf* classes), and the higher-level (node or container) class `'Emergency vehicle'`. This label selection ensures fine-grained class discrimination while maintaining backward compatibility with the AudioSet ontology.

To emulate realistic urban soundscapes, a broader set of *Negative* samples was selected to reflect typical vehicular and environmental backgrounds. These were grouped semantically and expanded into the following explicit AudioSet label lists:

- *Vehicle-related sounds:* `'Car'`, `'Car passing by'`, `'Power windows, electric windows'`, `'Tire squeal'`, `'Motor vehicle (road)'`, `'Truck'`, `'Air brake'`, `'Ice cream truck, ice cream van'`, `'Bus'`, `'Motorcycle'`, `'Skidding'`, `'Race car, auto racing'`, `'Bicycle'`, `'Train'`, `'Rail transport'`, `'Train wheels squealing'`, `'Railroad car, train wagon'`, `'Skateboard'`
- *Alarm and signal sounds:* `'Car alarm'`, `'Vehicle horn, car horn, honking'`, `'Bicycle bell'`, `'Train horn'`, `'Train whistle'`, `'Foghorn'`, `'Toot'`, `'Reversing beeps'`, `'Beep, bleep'`, `'Civil defense siren'`, `'Alarm'`, `'Smoke detector, smoke alarm'`, `'Fire alarm'`, `'Buzzer'`
- *Urban and environmental background (Traffic):* `'Traffic noise, roadway noise'`, `'Outside, rural or natural'`, `'Outside, urban or manmade'`
- *Additional distractors:* `'Speech'`, `'Conversation'`, `'Narration, monologue'`, `'Babbling'`, `'Music'`, `'Electric engine'`, `'Engine'`, `'Vehicle'`

To ensure semantic coherence and avoid contamination, a blacklist filter was applied to remove ambiguous or misaligned entries. Notably, the class `'Civil defense siren'` — despite its acoustic similarity — was excluded from the *Positives* due to its detachment from EV semantics, and assigned to the *Negatives*.

3.1.1 Positives curation

Positive samples were extracted through a three-step process:

1. *Label Filtering:* samples from the `balanced`, `unbalanced`, and `eval` splits were filtered based on the selected siren-related labels, yielding 146, 8154, and 148 entries respectively.
2. *Metadata Consolidation:* previous results were merged into a unified list (to simplify handling stages and increase content contributions) resulting in 8448 segments, with harmonized metadata and duplicates removal.
3. *Blacklist Filtering:* samples containing the `'Civil defense siren'` label were excluded, resulting in 8409 semantically consistent Positive segments.

All related statistics are logged in structured `.json` files (GitHub, Zenodo).

3.1.2 Negatives curation

The Negative set was constructed from a much larger and heterogeneous label space and involved an additional balancing stage:

1. *Label Filtering:* over 1.79M segments from all AudioSet splits were matched against the previous list of urban *distractors*.
2. *Metadata Consolidation:* these entries were unified into a consolidated pool of 1,797,595 de-duplicated segments.
3. *Partial Blacklisting:* to prevent label leakage, samples co-labeled with any target Positive class were discarded, except those labeled with `'Civil defense siren'`, which were retained by design. This yielded 1,794,186 Negative entries (Table 5).
4. *Balancing:* to correct severe class imbalance and reduce redundancy, a randomized downsampling procedure was applied. The selection was stratified to preserve the relative proportion of the main Negative sub-groups (e.g., vehicle sounds, alarms, speech, music, and general urban noise), ensuring that heterogeneous *distractors* remained represented. The final Negative set includes 8501 samples, maintaining both nominal balance (w.r.t. *Positives*) and intra-class diversity (Sect. 3.2 provides a more detailed analysis).

Table 3 AudioSet-EV metadata processing time using AudioSet-Tools

Step	Time (s)
Positives filtering	1.73
Positive segments merging	0.01
Positives blacklisting	0.07
Negatives filtering	3.98
Negative segments merging	2.94
Negatives blacklisting	9.15
Negatives rebalancing	9.99
Total processing time	**27.87**

Detailed label statistics and distributions are included in the public metadata (GitHub, Zenodo).

3.1.3 Download and post-processing

To provide a practical indication of the computational efficiency of our framework, we measured the time required to build the AudioSet-EV metadata structure on a standard workstation.[6] Table 3 reports the breakdown by processing steps. The overall processing required approximately 28 s, with the negative sample processing and rebalancing phases dominating the computational cost (due to the original AudioSet distribution size). These results confirm that the overhead introduced by the framework is limited, ensuring scalability to larger-scale distributions.

Metadata lists for *Positives* and *Negatives* were processed using two separate instances of the `AudioSet-Tools Downloader`. Each instance was configured to retrieve audio from YouTube at 32 kHz, convert stereo signals to mono, and skip loudness normalization. Time and error logs, reports, and download integrity checks were automatically generated. Since *Negative* balancing is stochastic, reproducible variants of the corpus can be created by changing the random seed. The complete dataset, including curated audio clips and metadata (`Seed`: 42), is distributed as a compressed `.zip` archive (8.3 GiB total).

3.2 Dataset structure and descriptive statistics

The `AudioSet-EV` corpus is structured into two main groups: *Positives*, including EV-related siren classes — `'Police car (siren)'`,`'Ambulance (siren)'`, `'Fire engine, fire truck (siren)'`, and the higher-level `'Emergency vehicle'` label — and *Negatives*, a deliberately heterogeneous set of non-EV categories spanning *vehicle sounds*, *alarm signals*, *speech*, *music*, and general *urban noise*.

Figure 4 shows that the *Positives* set, although limited in size, maintains high semantic coherence and acoustic consistency — even under weak labeling assumptions. In contrast, the *Negatives* set (Fig. 5) includes both taxonomically adjacent labels (e.g., `'Siren'`, `'Vehicle horn'`) and acoustically confounding classes (e.g., `'Engine'`, `'Alarm'`, `'Civil defense siren'`). This intentional diversity introduces *hard negative* samples that challenge trivial boundaries and promote discriminative representation learning.

The inclusion of `Speech` and `Music` enhances ecological validity by simulating acoustically cluttered urban soundscapes. Although no ablation study was conducted, their inclusion is theoretically motivated: interrogative pitch contours, vocal glissandi, or bowed musical tones can resemble the modulation patterns of *Wail* or *Yelp* sirens. Likewise, rhythmic or percussive pitched events may mimic stuttered or pulsed sirens, contributing to more robust generalization in real-world settings.

We computed label co-occurrence matrices to assess intra-group purity and inter-group separation. Figure 6 confirms the internal consistency of the *Positives* group. Over 96.4% of the samples include at least one EV-*leaf* label, and 89.2% exhibit multiple EV-related tags. The most frequent pairing is `'Siren'` and `'Emergency vehicle'` ($n = 3938$), highlighting the effectiveness of the hierarchical filtering strategy. It should be noted that some events belonging to the Negative group also appear in Figs. 6 and 7. This is not a contradiction, but a direct consequence of AudioSet's weakly labeled and multi-tag nature: a single audio clip may contain both EV-related sirens (retained in the *Positives* set) and co-occurring non-EV events such as speech, music, or alarms. These overlaps are intentionally preserved, as they reflect real-world acoustic conditions and provide challenging content factors that improve ecological validity and training robustness.

The *Negative–Traffic* subset (Fig. 7) reveals notable semantic heterogeneity. Labels such as `'Speech'` and `'Music'` dominate over core vehicular tags like `'Engine'` (12.1%) and `'Car'` (13.9%), reflecting a common issue in AudioSet: the foreground content often overshadows the background scene. Audio recorded via handheld devices may over-represent proximal speech, while melodic or gliding pitch patterns can be misinterpreted as musical elements — reinforcing weak-tag inconsistencies. Moreover YouTube contents are statistically dominated by human-related contents, rather than

[6] Reported times depend on the hardware configuration and are provided here only as reference values. The experiments were performed on an Intel®Xeon® 2.30GHz (8 cores) CPU, 64GB Kingston DDR4 RAM, NVIDIA ADA RTX6000 GPU (48GB RAM) — Driver v.550.52.15 — CUDA NV Cross Compiler v.12.5, 500GB Kingston NVMe SSD, running Linux Debian 13.1.0.

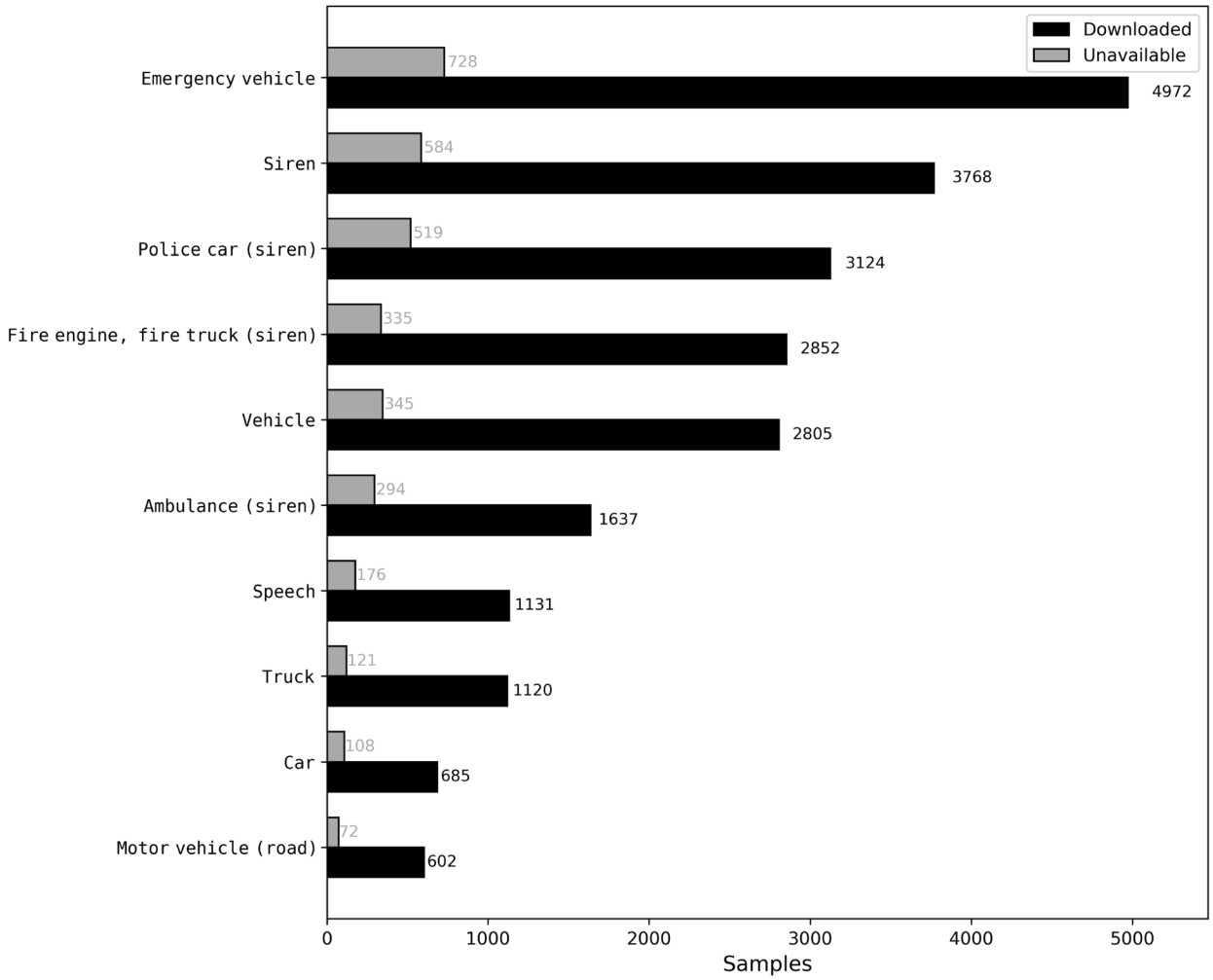

Fig. 4 Positives: top-10 label occurrences

free-field or traffic ones. These observations emphasize the limitations of weak labeling and the importance of task-specific corpus refinement.

The *Negatives–Alarms* matrix (Fig. 8) demonstrates better internal alignment, with strong co-occurrences among `'Alarm'`, `'Car alarm'`, and `'Vehicle horn'`. Still, some overlap with EV-related labels persists: `'Siren'` appears in 360 samples, and `'Police car (siren)'` in 338.

Finally, Fig. 9 shows the *Negatives–Vehicles* matrix. Consistent pairings such as `'Car'` and `'Motor vehicle (road)'` ($n = 7375$), or `'Bus'` and `'Truck'` ($n = 5284$) reveal strong internal coherence. However, the pervasive presence of `'Speech'` ($n = 13,520$) and `'Music'` ($n = 3896$) reinforces the challenges of disentangling relevant sounds from structured background noise.

Finer-grained analyses and statistics are available via dedicated Jupyter Notebooks and structured reports in our public repository (GitHub).

3.3 PyTorch/lightning AI utilities

To enable modular and reproducible training workflows, we developed a suite of PyTorch-based [49] utilities tailored to the structure of `AudioSet-EV`. At the core lies the `AudioSet_EV_Dataset` class, which abstracts the loading and pre-standardization of `.wav` audio files. Each sample is organized with one-hot encoding (0 for *Negatives*, 1 for *Positives*) and normalized to a 10-s duration via causal zero-padding or truncation. Unreadable or corrupted files are automatically excluded and logged (through `collate functions`) to ensure reproducibility.

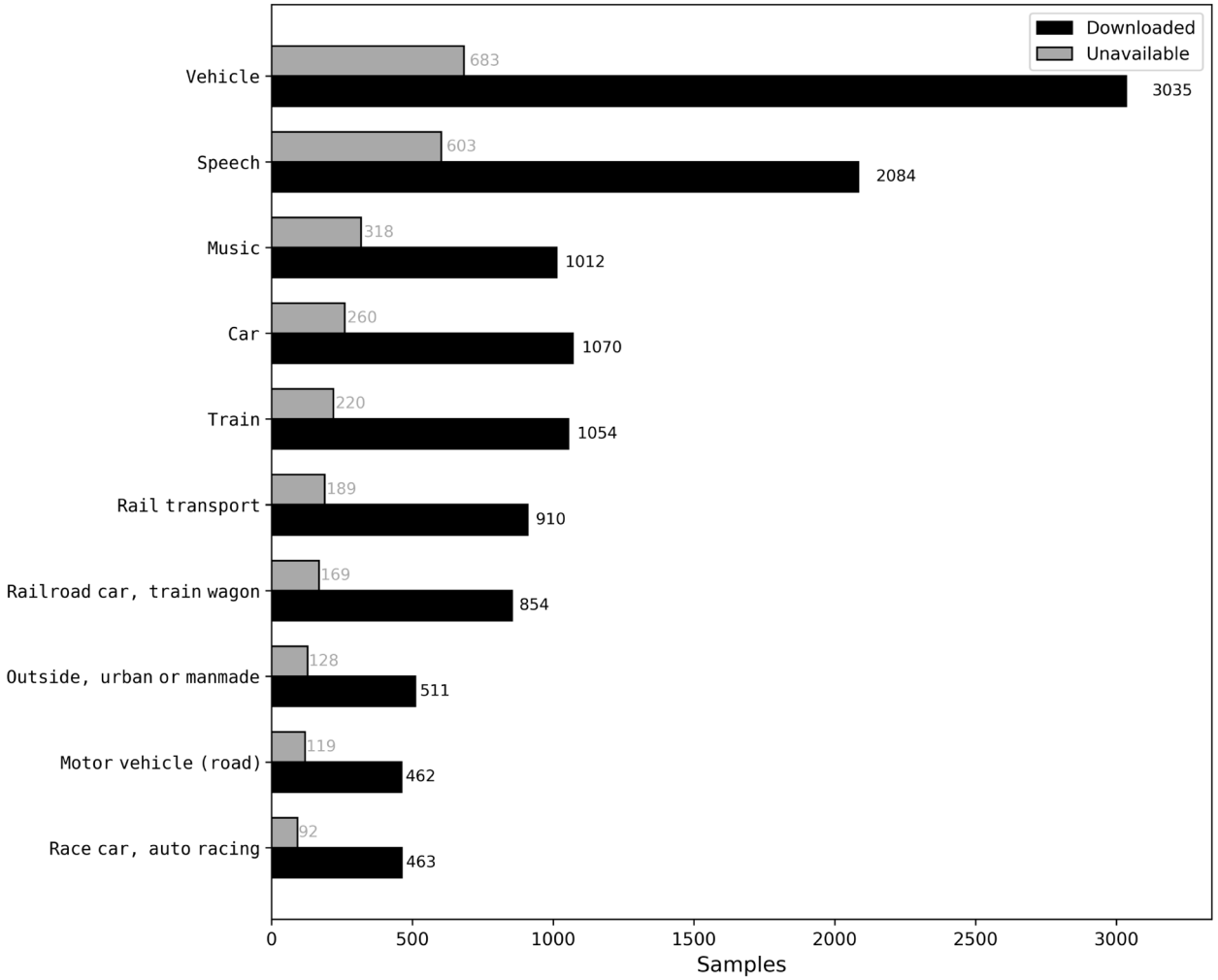

Fig. 5 Negatives: top-10 label occurrences

This dataset class is encapsulated within a PyTorch `DataModule` (`AudioSet_EV_DataModule`), which manages the initialization of train, validation, and test splits (default: 80%–10%–10%, fixed random seed), and implements efficient batch-wise loading through a custom `collate_function` and parallel workers. This function ensures consistent tensor shapes across mini-batches by dynamically padding sequences and discarding failed loads. The module supports multi-worker prefetching and shuffling to enable scalable workflows.

To simulate real-world acoustic variability and improve generalization, we introduced an augmented variant, `AudioSet_EV_Aug_DataModule`, implemented using PyTorch Lightning [50]. This version applies on-the-fly transformations, whose design and implementation details are presented in the next subsection. All modules and training utilities are publicly released in our GitHub.

3.4 AudioSet-EV augmentations

Spectral-domain augmentations such as SpecAugment [51], MixUp [52], spectro-temporal masking [53], and PatchOut [54] are widely adopted in DL pipelines. However, we deliberately excluded them from our external data preparation process due to their dependency on model-specific representations (e.g., spectrograms for CNNs, token sequences for Transformers). Embedding such augmentations within the training loop entangles data semantics with architecture design, hindering reproducibility and complicating cross-model benchmarking.

Instead, we adopt a modular, architecture-agnostic approach based on lightweight time-domain augmentations, implemented in the `AudioSetEV_Aug` class and applied *on-the-fly* during data loading stages. These transformations operate directly on the raw waveform and are fully configurable through external parameters. The

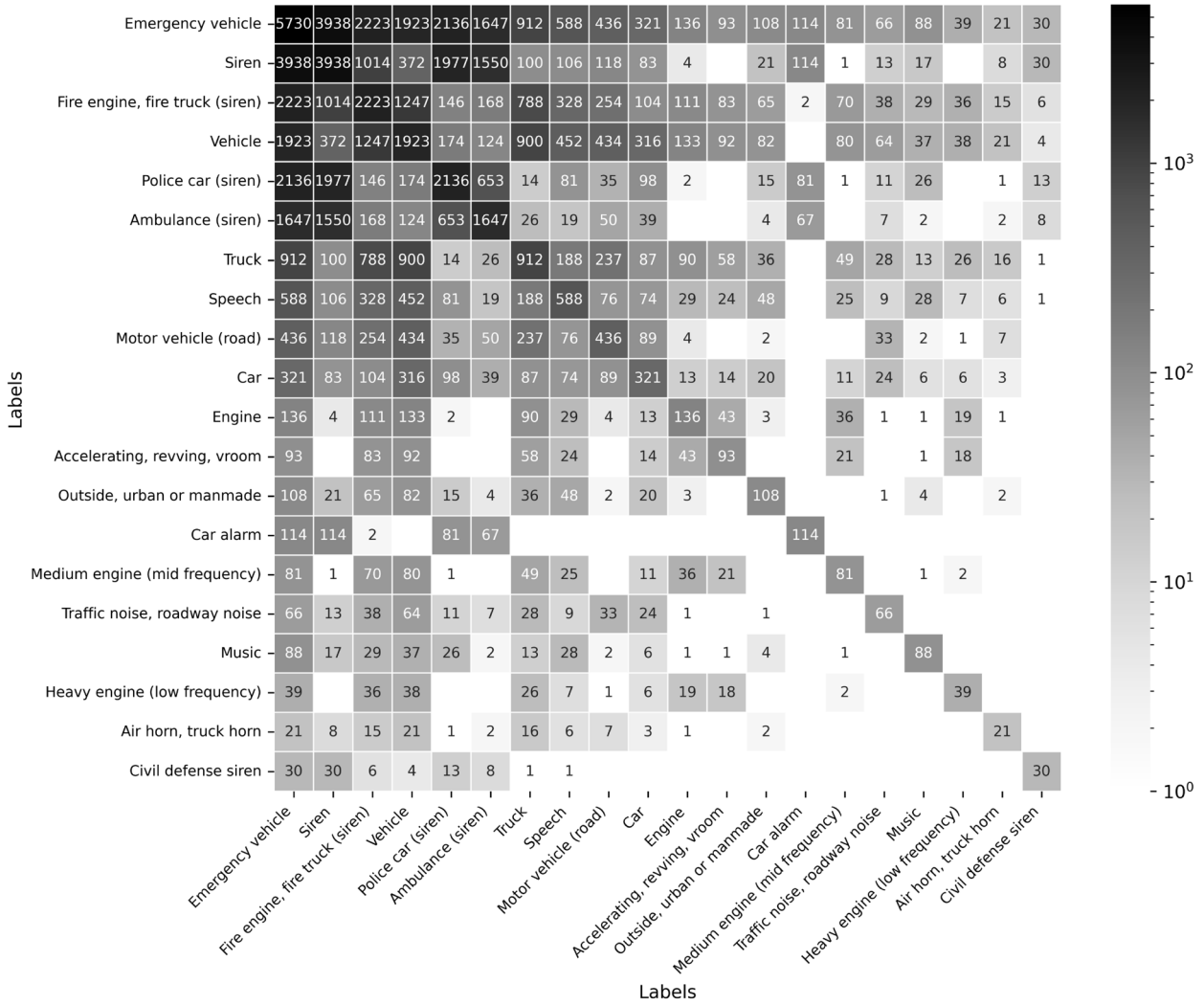

Fig. 6 *Positives* subset: top-20 label co-occurrences matrix

selected strategies simulate real-world acoustic variability (typical of urban environments), trying to enhance model robustness without compromising semantic validity.

The main techniques, given $x(t)$ a sample audio waveform and $\hat{x}(t)$ its augmented version, include:

3.4.1 Additive noise

Zero-mean noise is added to the input waveform to simulate sensor artifacts and mild background interference:

$$\hat{x}(t) = x(t) + \sigma \cdot \epsilon(t) \tag{1}$$

where $\epsilon(t) \sim \mathcal{U}(-1, 1)$ (uniform distribution) or $\mathcal{N}(0, 1)$ (Gaussian distribution with zero mean and unit variance), and σ is sampled in $[0.005,\ 0.05]$.

3.4.2 Random time shifting

Clips are circularly shifted to simulate variability in event onset:

$$\hat{x}(t) = x((t - \delta) \bmod T) \tag{2}$$

where $\delta \sim \mathcal{U}(0, T)$ is the random shift amount, $T = 10$ s is the clip duration, and mod denotes the modulo operator ensuring wrap-around continuity.

3.4.3 Polarity inversion

To introduce phase variation while preserving spectral content:

$$\hat{x}(t) = -x(t) \tag{3}$$

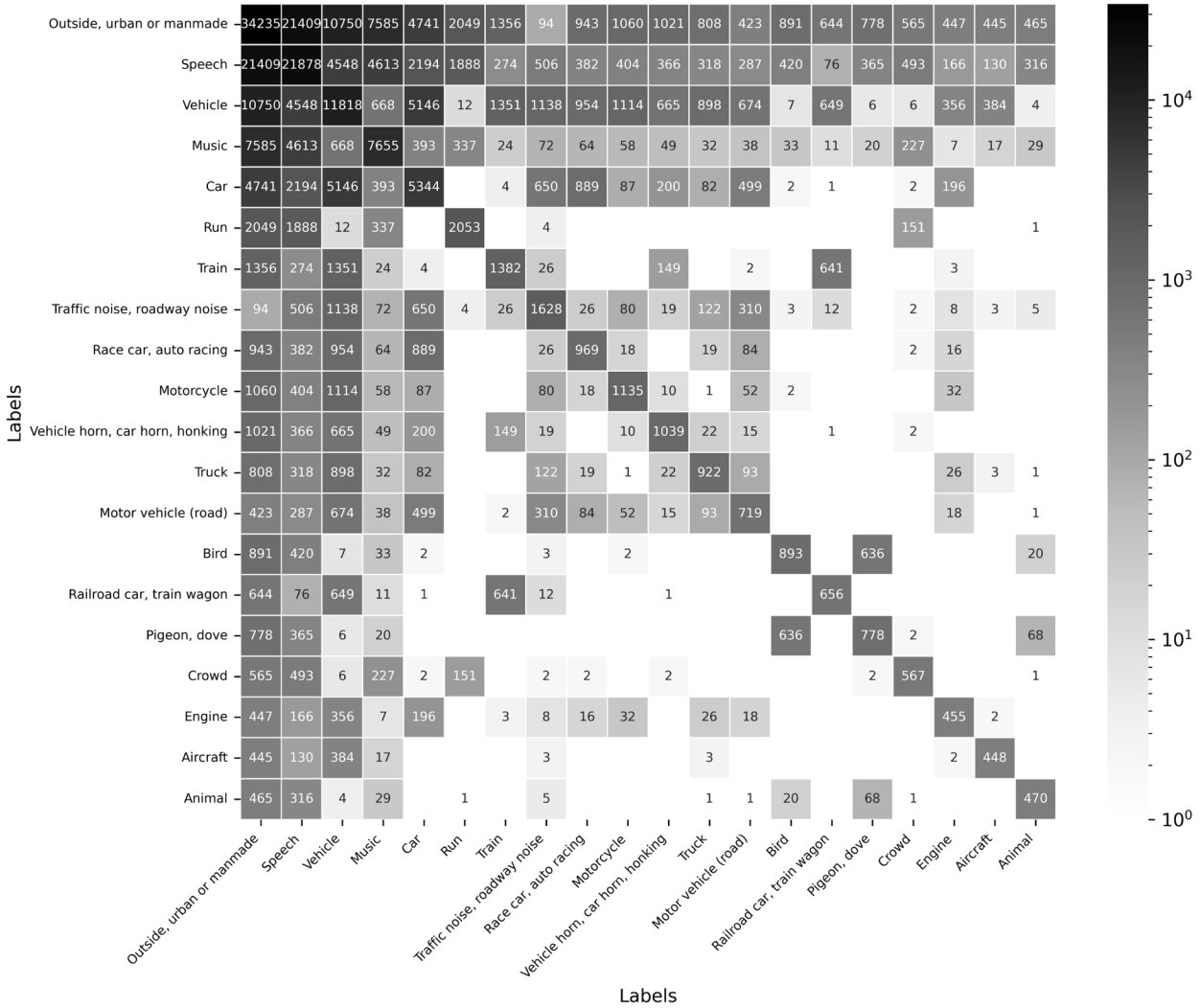

Fig. 7 Negatives–traffic: top-20 label co-occurrences matrix

This transformation is applied uniformly with a probability threshold $p = 0.5$.

3.4.4 Amplitude scaling

Two variants are used: global and local scaling. Global scaling simulates gain or distance variation:

$$\hat{x}(t) = \alpha \cdot x(t), \quad \alpha \sim \mathcal{U}(0.1, 1.0) \tag{4}$$

Local scaling applies time-varying amplitude scaling:

$$\hat{x}(t) = \alpha(t) \cdot x(t), \quad \alpha(t) \sim \mathcal{U}(0.01, 0.1) \tag{5}$$

where (a, b) denote the lower and upper bounds of the uniform distribution from which $\alpha(t)$ is sampled.

Augmentations are applied (Table 4) independently and may be chained in randomized sequences depending on task requirements. The full augmentation pipeline is executed prior to feature extraction and model ingestion, and supports reproducible runs by logging applied parameters per batch. This strategy enhances generalization in complex acoustic scenes while preserving semantic control over the dataset.

4 Results and discussion

This section presents a multi-level evaluation framework aimed at assessing the consistency, informativeness, and practical utility of the proposed `AudioSet-EV` dataset for EV siren recognition. The analysis is articulated into three complementary benchmark activities, designed to validate the dataset from quantitative, qualitative, and application-oriented perspectives.

First, we introduce a dataset-level benchmark, systematically comparing `AudioSet-EV` to a curated set of publicly available corpora featuring EV-related acoustic

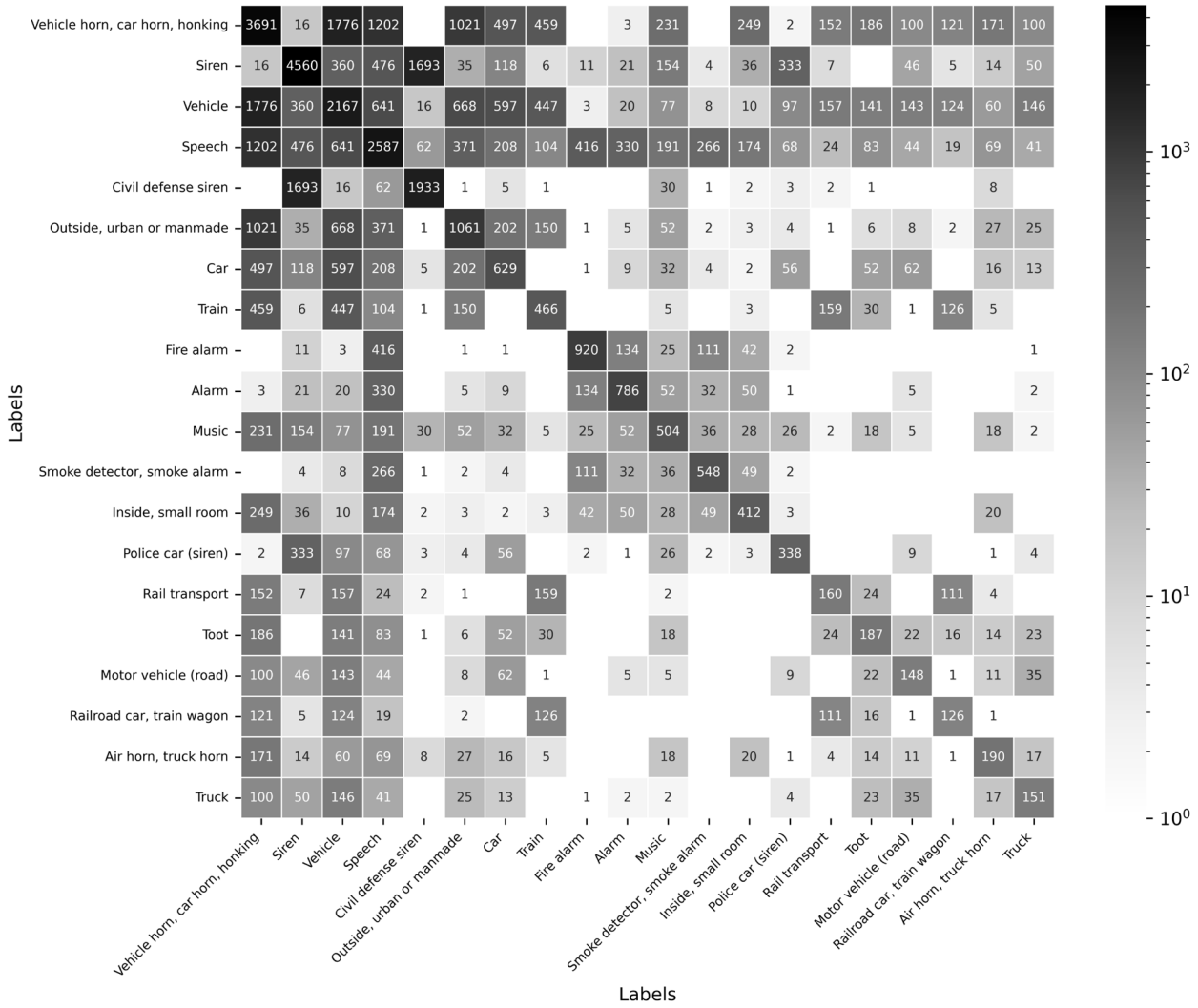

Fig. 8 Negatives–alarms: top-20 label co-occurrences matrix

events. These include *SireNNet* [27], *LSSiren* [28, 29], *ESC-50* [30], *UrbanSound8K* [21], and *FSD50K* [22], each characterized by varying annotation detail, taxonomic structure, background variability, and audio quality. To enable a fair and reproducible comparison, all datasets are integrated within a harmonized evaluation pipeline based on our PyTorch-Lightning design (Sect. 3.3). This pipeline enforces consistent pre-processing, labeling, and batching procedures via custom `Dataset` and `DataModule` classes. Standardization steps include the following: (1) one-hot encoding of labels (siren vs. non-siren), (2) resampling to 32 kHz (matching AudioSet), (3) waveform trimming or padding to a fixed contents duration, and (4) mono signals reduction for channel consistency. Dataset-specific splits and evaluation protocols (e.g., cross-validation or predefined test sets) are preserved, while uniform `collate functions` ensure runtime compatibility by parsing audio files, applying pre-processing when required, and assembling input batches with homogeneous dimensionality for model training. Taxonomic harmonization is achieved through the Standardized Audio Event Label Taxonomy (SALT) [23], whose Python API [26] supports automatic mapping between dataset-specific labels and shared semantic descriptors. This enables quantification of label intersections, overlaps, and semantic drift (discrepancies that occur when nominally similar labels, across datasets, are not perfectly aligned in their taxonomic scope or annotation criteria), providing a structured basis for cross-dataset benchmarking[7].

[7] Implementation details on how dataset-specific metadata and labels were parsed, mapped, and encoded for cross-dataset analysis are available on our (GitHub).

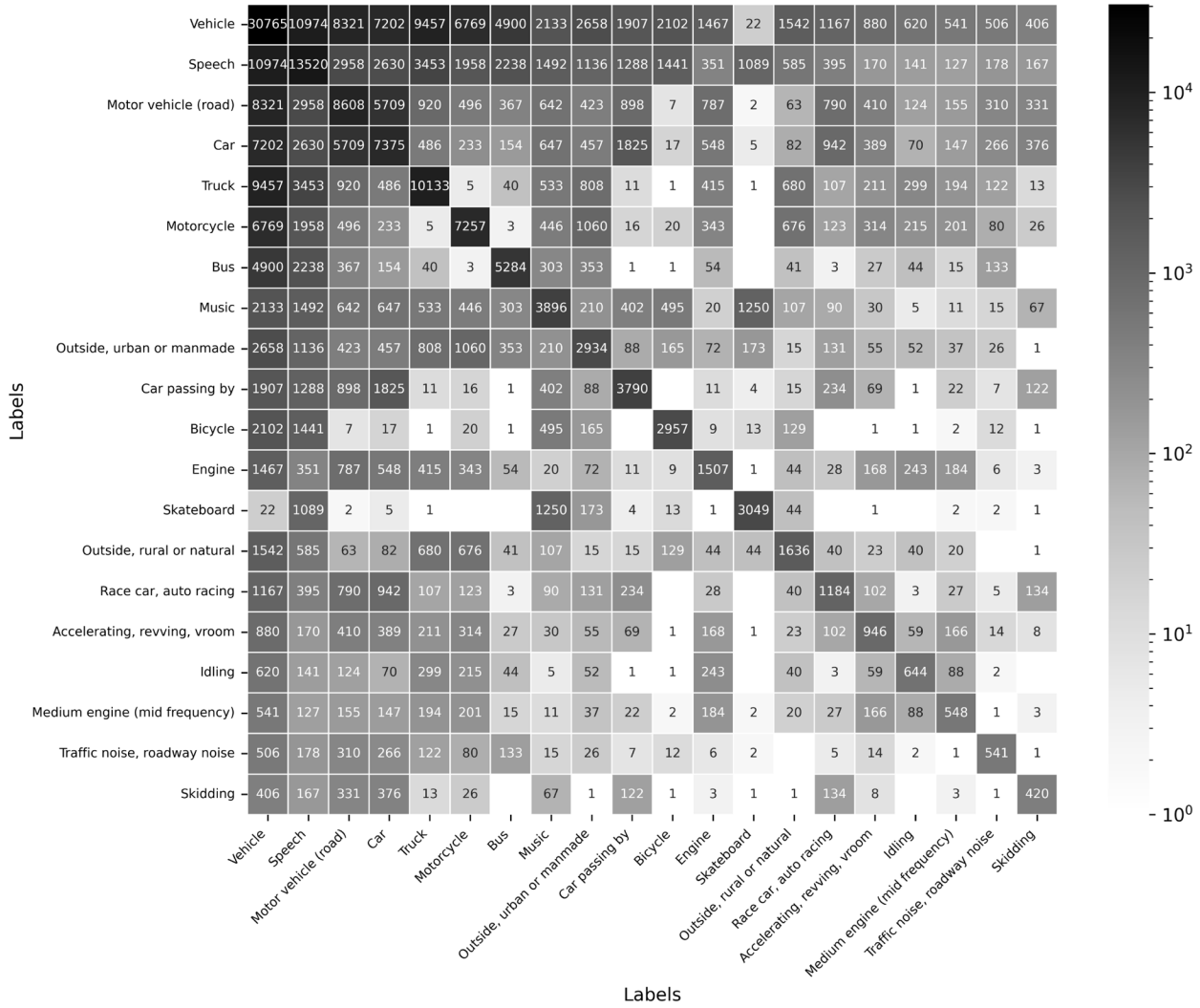

Fig. 9 Negatives–vehicles: top-20 label co-occurrences matrix

Table 4 Default parameters in *AudioSetEV_Aug*

Augmentation	Default parameters
Additive noise	$\sigma = 0.1, \epsilon(t) \sim \mathcal{U}(-1,1)$ or $\mathcal{N}(0,1)$
Time shifting	$T = 10\,\text{s}, \delta \sim \mathcal{U}(0,T)$
Polarity inversion	Applied with probability $p = 0.5$
Global scaling	$\alpha \sim \mathcal{U}(a = 0.1, b = 1.0)$
Local scaling	$\alpha(t) \sim \mathcal{U}(a = 0.01, b = 0.1)$

Second, we evaluate the practical utility of `AudioSet-EV` as a training corpus through a model-centric benchmark. Specifically, we fine-tune the efficient pre-trained audio neural networks (E-PANNs) model [55], a pruned and latency-efficient version of the original CNN-14 architecture from the PANNs family [56]. Designed for general-purpose audio tagging on AudioSet, E-PANNs balances compactness and accuracy via two core pruning strategies: (1) a data-driven, singular value decomposition (SVD)-based technique [57] that removes low-rank convolutional filters with minimal contribution to feature variance, and (2) an operator-norm-based filter ranking method [58] that selects structurally relevant kernels in a dataset-agnostic fashion. This dual strategy enables fine-grained model compression without compromising performance, making E-PANNs suitable even for real-time sound event detection (SED) applications. We turn E-PANNs into a binary EV-siren classifier, using `AudioSet-EV` as the sole training source. The fine-tuned model is then evaluated on all datasets in the benchmark suite using their native testing protocols. This setup allows us to assess the robustness of representations learned from

AudioSet-EV when transferred across corpora with heterogeneous annotation practices and recording conditions. In fact, although all datasets are normalized to a binary label space (*siren* vs. *non-siren*), they substantially differ in terms of annotation methodology (manual vs. crowdsourced, strong vs. weak, fine-grained vs. coarse labels) and in their acoustic provenance (e.g., YouTube, Freesound, urban field recordings, or ad hoc siren collections), indirectly influencing the resulting data distributions. These differences provide a natural source of variability for testing cross-dataset generalization and mitigate overfitting to the specific biases of any single corpus.

Finally, we close the loop with a *bi-directional evaluation* stage. While the previous benchmarks focus on assessing the transferability of a model trained on AudioSet-EV to external corpora, here we perform the reverse experiment: training the same E-PANNs architecture independently on each external dataset, and subsequently testing on the AudioSet-EV test split. This setup provides a complementary perspective, enabling us to quantify to what extent existing corpora capture the variability and acoustic diversity embedded in AudioSet-EV. The comparison highlights whether generalization gaps are symmetrical or whether AudioSet-EV plays a unique role as a semantically controlled and structurally balanced resource for EV siren recognition.

4.1 Comparison with existing EV benchmarks

To standardize the taxonomical alignment across corpora, we employed the Standardized Audio Events Label Taxonomy (SALT) [23] and its Python interface [26]. This enabled us to systematically map class correspondences between the AudioSet-EV taxonomy and the native label sets of each benchmark dataset, resolving ontological inconsistencies for comparative evaluation. Figure 10 illustrates the resulting alignment for emergency_vehicle and siren_ringing labels across datasets.

Each dataset was integrated into our evaluation pipeline through corpus-specific procedures, briefly summarized below:

- **SireNNet** was binarized into EV vs. non-EV classes as per its original design [27]. We replicated its

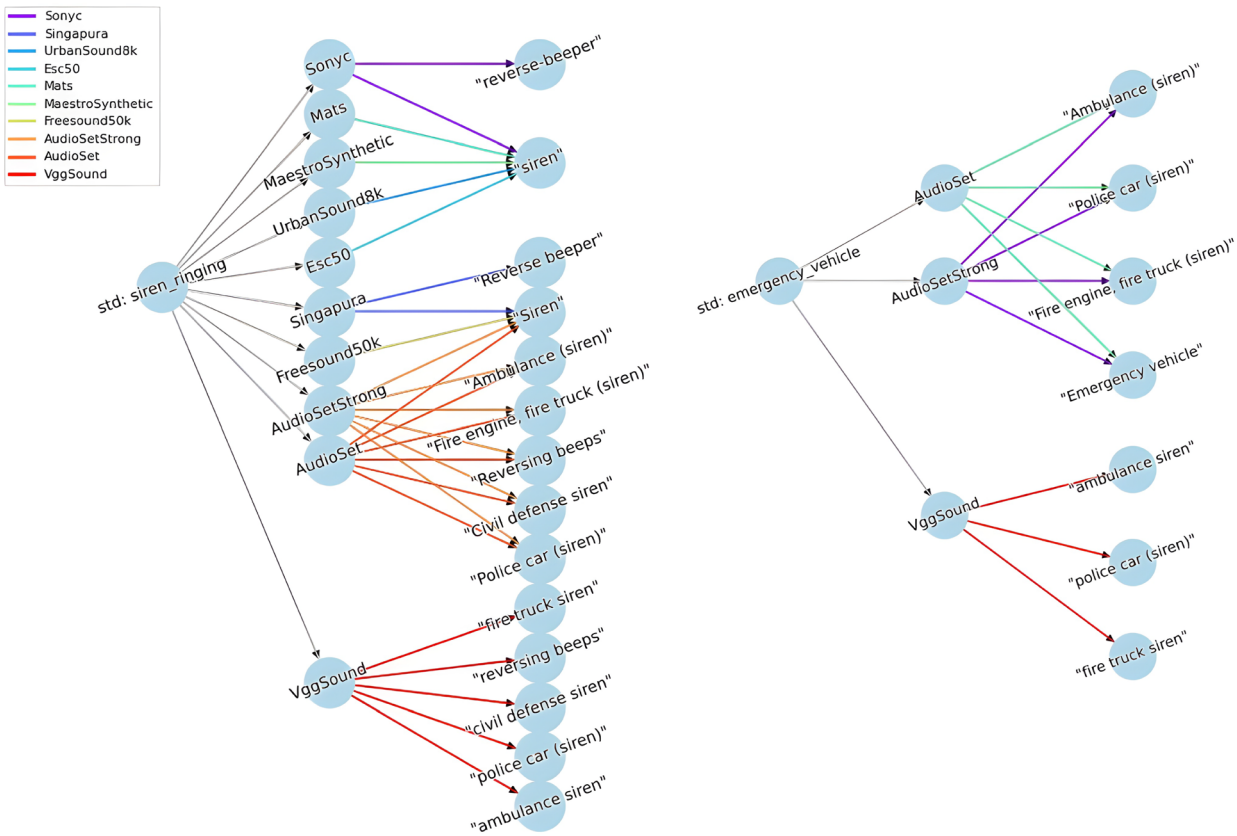

Fig. 10 SALT-based mapping of *emergency_vehicle* and *siren_ringing* labels across benchmark datasets

scalable evaluation scheme by generating random, balanced subsets of increasing size.

- **LSSiren** samples were zero-padded to a minimum duration of 1 s at 32 kHz, enabling mini-batch training. A custom `collate function` handles dynamic padding at loading time.
- **ESC-50** includes the label `siren` among other urban classes. We isolated siren instances as positives and drew negatives from remaining urban sounds. The original 5-fold cross-validation was preserved.
- **UrbanSound8K** was binarized similarly, treating `siren` as positive and applying fold-wise evaluation per the authors' guidelines [21].
- **FSD50K** required custom filtering based on SALT. We extracted `siren` samples as positives and selected diverse urban classes as negatives. Metadata were exported in `.csv` format compatible with our `Test_Dataset` interface (Sect. 3.3), using the official `eval` split [22].

All datasets were harmonized in format and evaluation strategy while preserving their native structure and distributions. This allowed comparison along both technical and taxonomic dimensions, as summarized in Tables 5 and 6.

The distribution of samples confirms that `AudioSet-EV` contains the largest number of labeled segments, both in its *raw* and *balanced* forms (Table 5). Replicating the same downsampling procedure to balance other datasets would lead to severe reductions in usable data, compromising their practical applicability. For instance, *ESC-50*'s test sets contain only 72 samples per fold (roughly 6 min), with an 1:8 imbalance between positive and negative classes — limiting statistical significance for most performance metrics. In contrast, *SireNNet* introduces a variable-split evaluation framework with multiple configurations ranging from 4 to 1258 samples, reaching a 2:1 class ratio at best. However, its total content duration remains modest due to the 3-s fixed clip length. *LSSiren* is the only benchmark specifically balanced for EV siren recognition, but its size is nearly one order of magnitude smaller than `AudioSet-EV`. Moreover, its test splitting strategy is not explicitly defined, and the original sampling rate is unclear (with reports of custom upsampling to 48 kHz in [29]), while durations may be inflated due to padding strategies used during training. Similarly, *UrbanSound8K* and *FSD50K* apply padding post-resampling, to harmonize audio lengths during batch collation, leading to slight over-estimations of duration. *FSD50K*, while offering the longest total audio content (1773 min), suffers from an extreme imbalance (55 positives vs. 3893 negatives), approximately 1:71.

Table 5 EV-benchmark dataset samples distribution

Dataset	Positive samples	Negative samples	Total
AudioSet-EV (unbalanced)	8409	1,794,186	1,802,595
AudioSet-EV (balanced)	**8409**	**8501**	**16,910**
ESC-50	40	320	360
SireNNet	1254	421	1675
LSSiren	932	902	1834
UrbanSound8K	929	7803	8732
FSD50K	133	21,844	21,977

By contrast, the `AudioSet-EV` test split (10% of dataset) offers 1404 10-s mono clips at 32 kHz, balanced across 752 positives and 652 negatives for a total of 234 min. The full dataset retains 90% of its data for training and evaluation, supports customizable splits, and offers a flexible basis for resampling, re-segmentation, and downstream annotation.

4.1.1 Qualitative analysis: class coverage, balance, and coherence

From a taxonomic standpoint, existing benchmark datasets present notable limitations. Neither *ESC-50* nor *UrbanSound8K* includes fine-grained mappings to `Emergency vehicle` or its leaf categories in AudioSet. Due to the constraints of SALT, applicable labels in these corpora were mapped to the broader `siren` class, aggregating semantically related urban sounds. While pragmatic, this strategy underscores the semantic advantage of `AudioSet-EV`, which explicitly targets emergency siren categories and couples them with a diverse, ecologically valid set of negative samples.

Figures 4 and 5 validate the effectiveness of our curation pipeline. The Top-5 positive classes extracted via `AudioSet-Tools` correspond to the design targets defined in the filtering stage, confirming the correctness of the implemented procedures. The only minor discrepancies observed between pre-filtering label counts and post-filtering outcomes are attributable to content unavailability on YouTube at the time of processing. Indeed, our experiments relied on the official AudioSet metadata snapshot released in November 2024, and natural fluctuations in online availability (e.g., removed or restricted videos) inevitably affect the final sample counts. The *Negative* subset includes a well-distributed mix of urban environments, speech, and music — essential for training robust models that

Table 6 EV-test datasets (splits) benchmark

Part I – AudioSet-EV, ESC-50, SireNNet			
Test dataset	**AudioSet-EV (*ours*)**	**ESC-50**	**SireNNet**
Samples	**1 404**	72 × 5	[4, 8, 16, 32, 66, 134, 268, 536, 957, 1 258]
Positives	**752**	8 × 5	[2, 4, 8, 16, 33, 67, 134, 268, 536, 837]
Negatives	**652**	64 × 5	[2, 4, 8, 16, 33, 67, 134, 268, 421, 421]
Duration (min)	**234**	6 × 5	[0.2, 0.4, 0.8, 1.6, 3.3, 6.7, 13.4, 26.8, 47.85, 62.9]
Sample Dur. (sec.)	**10**	5	3
Audio Chs	**1**	1	1
Sampling rate	**32 KHz**	16 KHz	44.1 KHz
Test strategy	**10% split**	5×cross-val folders	10×varying-size splits
Part II – LSSiren, UrbanSound8K, FSD50K			
Test dataset	**LSSiren**	**UrbanSound8K**	**FSD50K**
Samples	1 834	[873, 888, 925, 990, 936, 823, 838, 806, 816, 837]	3948
Positives	449	[86, 91, 119, 166, 71, 74, 77, 80, 82, 83]	55
Negatives	932	[787, 797, 806, 824, 865, 749, 761, 726, 734, 754]	3893
Duration (min)	902	[58.2, 69.2, 61.67, 66, 62.4, 54.87, 55.87, 53.75, 54.4, 55.8]	1773.43
Sample Dur. (s)	Variable	Variable	Variable
Audio Chs	1	Variable	1
Sampling rate	Variable (48KHz*)	Variable	44.1KHz
Test strategy	*Unknown splits*	10×fold splits	eval split

* indicates that the corresponding value is not fully certain and has been taken directly from the original paper, as some files actually have a different sample rate

can generalize across complex, overlapping acoustic scenes. This is especially relevant in real-world deployments, where sirens may co-occur with speech or tonal foreground sources, such as street music, that challenge classification reliability.

Figure 11 shows high preservation rates (post-download) for dominant leaf classes, such as `Cars & light vehicles`, `Sirens & Civil alarms`, and `Speech & conversation`, with 80–86% of samples retained. The only significant drop is seen in `Vehicle alarms`

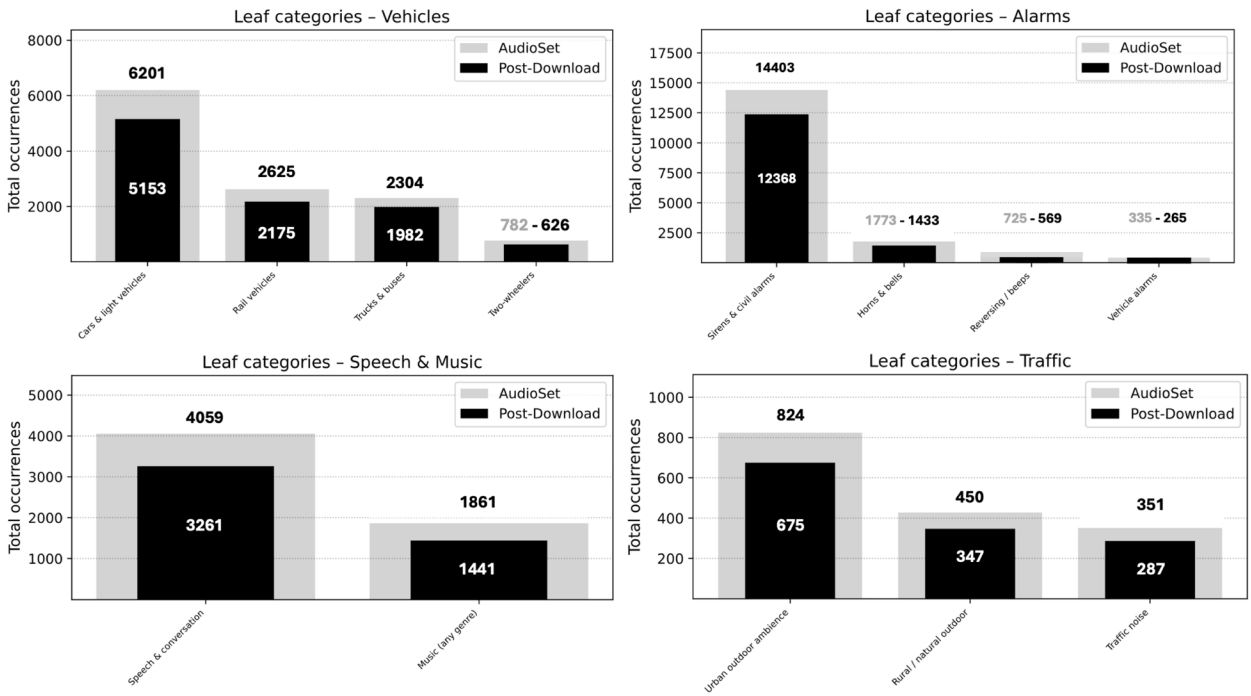

Fig. 11 Leaf-level class preservation across four major *AudioSet-EV* blocks. Grey: original AudioSet-EV counts; **Black**: post-download

(61%), likely due to platform-specific issues. Crucially, intra-block ranking remains stable, indicating that no structural distortions were introduced during the download process.

As shown in Fig. 12, the long-tailed nature of *Positives* is preserved. The most frequent labels remain stable up to rank 60, while data loss is limited to the extreme tail — consistent with *Zipf*-like decay patterns in open-domain corpora. The 100-samples threshold, representing roughly 1.6% of the dataset, offers a convenient benchmark: below this line, classes contribute minimally to training and can be considered statistically marginal. This natural filtering helps concentrate model learning on semantically relevant and frequently occurring classes.

The bottom plot of Fig. 12 reveals a broader, yet still well-structured, distribution of negative labels. This outcome reflects the intentional heterogeneity introduced during *Negative* subset curation, resulting in a preserved long-tail distribution spanning a more diverse semantic spectrum, ultimately fostering model robustness by exposing it to a wide range of non-EV but contextually relevant acoustic events.

Figure 13 complements this view with a node-level comparison. The overall post-download scaling factor remains close to 0.85 across major taxonomy nodes, demonstrating that the structural balance of the original corpus is preserved. In both AudioSet-native (upper) and SALT-remapped views (lower), key macro-categories like `Alarms` and `Vehicles` retain their dominance, confirming that thematic focus on EV-related events is maintained. Colored highlights in Fig. 13 trace minor redistribution in relevant macro-areas (e.g., `Vehicles`, `Speech, and Human`), consistent with design objectives and the taxonomic mapping process.

Rarity preservation is quantified in Fig. 14, which compares retained class counts below fixed sample-count thresholds (2, 5, 10, 20, 50, 100). The downloaded subset closely mirrors the original distribution, especially in the

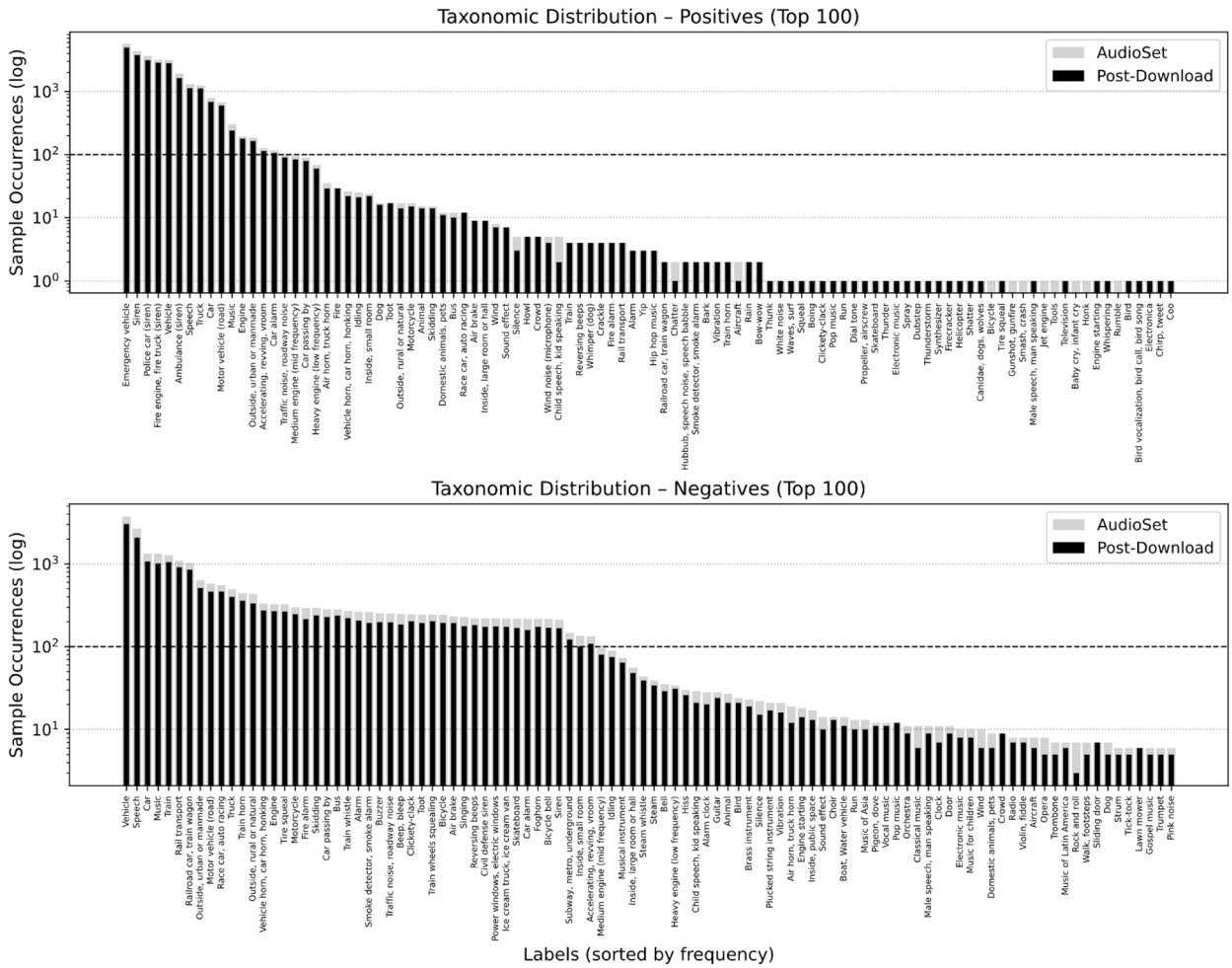

Fig. 12 Top-100 class distributions: positives (top), negatives (bottom). Grey: original AudioSet-EV; **Black**: downloaded subset. Dotted line: 100 samples

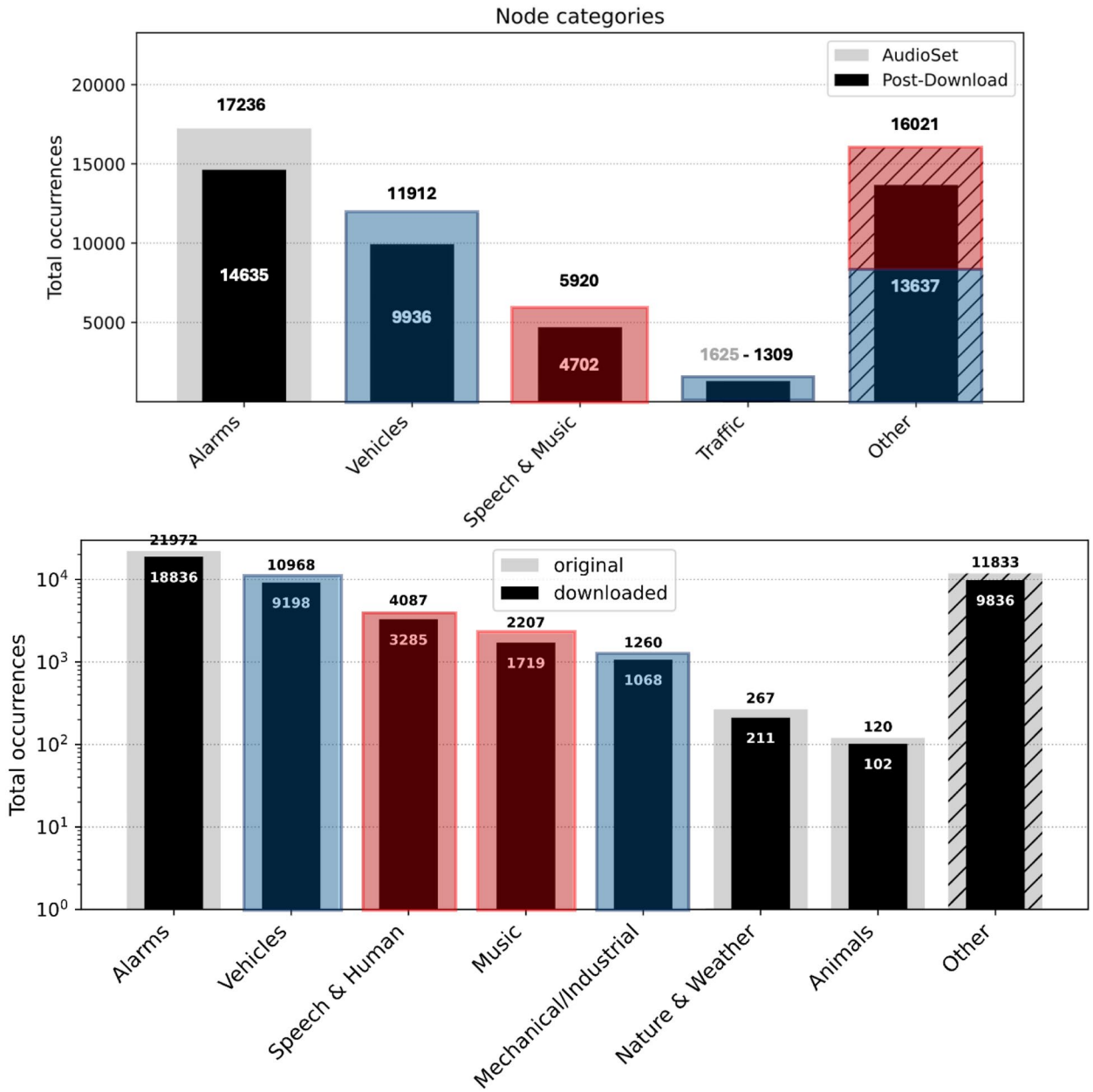

Fig. 13 AudioSet-EV node-level taxonomy comparison: AudioSet (top) and SALT (bottom). Colors indicate macro-categories where sample proportions changes, according to the reference taxonomy. Blue for `Vehicles`-related sounds, and Red for human-related sounds

Positives, where a stable pseudo-linear trend is evident. This indicates that semantically important rare classes were retained, while peripheral or noisy entries were excluded, ensuring both representational fidelity and training reliability.

In summary, our curated `AudioSet-EV` dataset (Zenodo) achieves a strong balance between semantic coherence and statistical robustness. Its taxonomic integrity, long-tail structure, and ecologically valid label design make it a valuable candidate for scalable and interpretable EV-siren recognition tasks. The results reaffirm the flexibility and rigor of the `AudioSet-Tools` framework in enabling precise, reproducible dataset curation from the broader AudioSet repository.

4.2 Accuracy evaluation

To assess the robustness and generalization of `AudioSet-EV`, we trained our baseline model on its curated

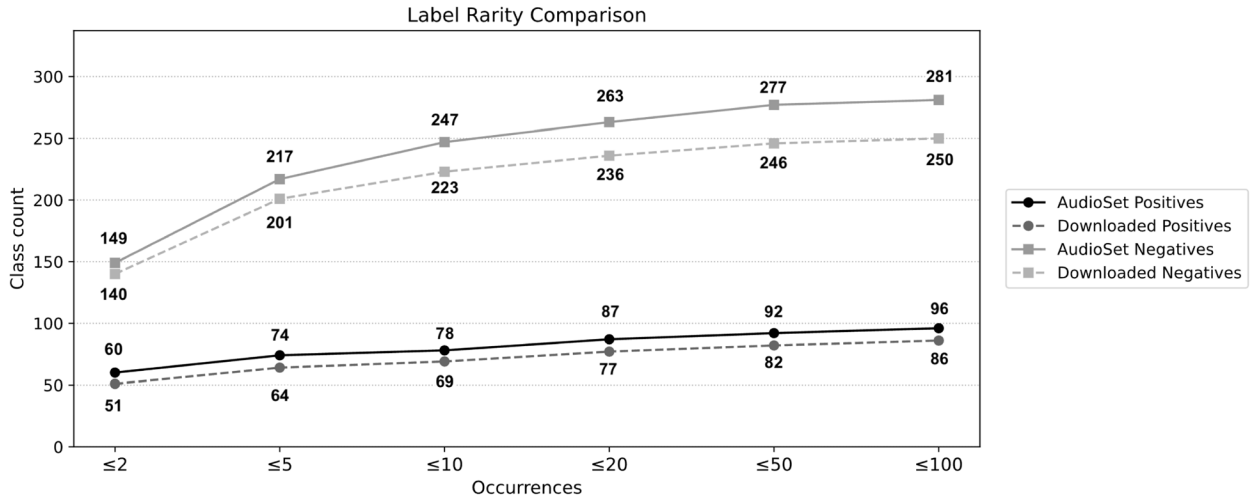

Fig. 14 Retention of rare classes across thresholds. *Positive* and *Negative* subsets

training subset, and evaluated it across all other available EV benchmarks (Sect. 4.1). The training pipeline adopted the same configuration used during dataset curation, including consistent audio pre-processing, batch-wise waveform collation with dynamic padding, and GPU-accelerated optimization via PyTorch Lightning. Input waveforms were resampled to 32 kHz to match model expectations. Datasets without predefined splits were partitioned using an 80%/10%/10% train/validation/test policy, in line with the `AudioSet-EV` protocol.

All datasets were processed using our unified dataloader architecture (Sect. 3.3), which retains dataset-specific durations while conforming to a common inference interface. Evaluation followed a binary classification protocol with a threshold $\theta = 0.5$, reporting accuracy, precision (PPV), recall (TPR), specificity (TNR), F_1 and F_2 scores, area under ROC and PR curves (AuROC, AuPRC), and Matthews correlation coefficient (MCC).

As shown in Table 7, the E-PANNs classifier trained on `AudioSet-EV` achieves strong and balanced performance across all evaluated domains. On its native test set, it surpasses 90% in accuracy, precision, and specificity, with F_1 and F_2 scores around 90%. These results confirm the model's ability to discriminate EV sirens from acoustically similar but irrelevant events — such as speech, alarms, or horns — with low false positive rates, which is essential in real-world monitoring contexts.

Performance on the `AudioSet-EV Augmented` test set remains high, particularly in precision (91.2%), despite the expected drop in recall (82.3%) due to added distortions. This asymmetry aligns with use cases where minimizing false alarms is prioritized over exhaustive detection.

The model's generalization is further validated on `FSD50K`, where it maintains 98.8% accuracy. Although recall remains limited (40%), the overall MCC of nearly 49% reflects meaningful discriminative capacity despite label mismatch and domain shift.

On `LSSiren`, the classifier achieves near-saturation performance (recall 98.9%, MCC 95.4%), demonstrating strong intra-class discrimination on a focused siren dataset. Results across `ESC-50` and `US8K` confirm consistent behavior: cross-validation on `ESC-50` yields F_1 and MCC scores above 94%, while on `US8K` the model maintains an average F_1 of 84.9% and MCC of 83.5%, with slight fold-wise variability.

The most consistent and high-impact results are observed on `SireNNet`, where across ten evaluation loaders the model achieves an average F_1 of 98.4% and MCC of 96.7%, confirming its applicability in custom deployment settings. Such performance demonstrates that training on a structurally balanced and semantically targeted corpus like `AudioSet-EV` enables effective generalization to diverse benchmarks and label structures.

4.2.1 Bi-directional validation

To further validate the generalization quality of our dataset, we extended the model evaluation protocol in a bi-directional fashion. In addition to training E-PANNs on `AudioSet-EV` and testing on external corpora, we reversed the process by training the same architecture (initialized with random weights) on each dataset in the EV-Benchmark suite and evaluating its performance on the held-out test split of `AudioSet-EV`. Hyperparameters were kept identical to those adopted for the

Table 7 E-PANNs uni-directional EV-benchmark

Dataset	Accuracy	Precision (PPV)	Recall (TPR)	F_1 Score	F_2 Score	Specificity (TNR)	AuROC	AuPRC	MCC
AudioSet EV	**90.5%**	**91.6%**	**89.7%**	**90.6%**	**90.9%**	**91.4%**	**90.5%**	**87.4%**	**81.1%**
AudioSet EV augmented	**86.9%**	**91.2%**	**82.3%**	**86.5%**	**87.5%**	**91.7%**	**87.0%**	**84.1%**	**74.2%**
FSD50K	**98.81%**	**61.11%**	**40.00%**	**48.35%**	**42.97%**	**99.64%**	**69.82%**	**25.28%**	**48.88%**
LSSiren	**97.71%**	**96.65%**	**98.93%**	**97.77%**	**98.46%**	**96.45%**	**97.69%**	**96.15%**	**95.44%**
ESC50 Fold 1	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
ESC50 Fold 2	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
ESC50 Fold 3	95.83%	85.71%	75.00%	80.00%	76.92%	98.44%	86.72%	67.06%	77.90%
ESC50 Fold 4	98.61%	88.89%	100.00%	94.12%	97.56%	98.44%	99.22%	88.89%	93.54%
ESC50 Fold 5	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Cross-Validation	**98.89%**	**94.92%**	**95.00%**	**94.82%**	**94.90%**	**99.38%**	**97.19%**	**91.19%**	**94.29%**
SireNNet Loader 0	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
SireNNet Loader 1	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
SireNNet Loader 2	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
SireNNet Loader 3	96.88%	100.00%	93.75%	96.77%	94.94%	100.00%	96.88%	96.88%	93.93%
SireNNet Loader 4	98.48%	100.00%	96.97%	98.46%	97.56%	100.00%	98.48%	98.48%	97.01%
SireNNet Loader 5	97.76%	98.48%	97.01%	97.74%	97.31%	98.51%	97.76%	97.04%	95.53%
SireNNet Loader 6	97.76%	99.23%	96.27%	97.73%	96.85%	99.25%	97.76%	97.39%	95.56%
SireNNet Loader 7	96.83%	99.22%	94.40%	96.75%	95.33%	99.25%	96.83%	96.46%	93.77%
SireNNet Loader 8	98.01%	99.43%	97.01%	98.21%	97.49%	99.29%	98.15%	98.13%	96.02%
SireNNet Loader 9	97.62%	99.63%	96.77%	98.18%	97.33%	99.29%	98.03%	98.56%	94.80%
Average Results	**98.33%**	**99.60%**	**97.22%**	**98.38%**	**97.68%**	**99.56%**	**98.39%**	**98.29%**	**96.66%**
US8K Fold 1	97.14%	85.06%	86.05%	85.55%	85.85%	98.35%	92.20%	74.56%	83.96%
US8K Fold 2	95.16%	77.91%	73.63%	75.71%	74.44%	97.62%	85.62%	60.06%	73.06%
US8K Fold 3	95.57%	89.80%	73.95%	81.11%	76.66%	98.76%	86.35%	69.76%	79.10%
US8K Fold 4	98.18%	93.53%	95.78%	94.64%	95.32%	98.67%	97.22%	90.29%	93.56%
US8K Fold 5	97.86%	88.06%	83.10%	85.51%	84.05%	99.08%	91.09%	74.46%	84.40%
US8K Fold 6	96.35%	89.29%	67.57%	76.92%	71.02%	99.20%	83.38%	63.24%	75.84%
US8K Fold 7	96.66%	91.53%	70.13%	79.41%	73.57%	99.34%	84.74%	66.93%	78.44%
US8K Fold 8	97.64%	86.75%	90.00%	88.34%	89.33%	98.48%	94.24%	79.06%	87.05%
US8K Fold 9	98.65%	93.83%	92.68%	93.25%	92.91%	99.32%	96.00%	87.70%	92.50%
US8K Fold 10	97.73%	88.10%	89.16%	88.62%	88.94%	98.67%	93.92%	79.62%	87.36%
Cross-Validation	**97.09%**	**88.39%**	**82.21%**	**84.91%**	**83.21%**	**98.75%**	**90.48%**	**74.57%**	**83.53%**

native `AudioSet-EV` training (available in our GitHub), including the binary classification protocol. Dataset-specific partitions into training and validation sets follow the proportions summarized in Table 8, with a batch size fixed to 32 samples, across all experiments. Table 9 reports the results of this bi-directional cross-dataset evaluation.

The evaluation globally highlights the severe limitations of training EV siren classifiers on external corpora and testing them on `AudioSet-EV`. Models trained on *SireNNet* and *LSSiren* achieve nearly perfect recall (close to 100%), but this comes at the cost of an almost total collapse in specificity (down to < 1%), yielding random-level ROC/PR performance (equivalent to a trivial

Table 8 Train/validation split sources and proportions for bi-directional evaluation (batch size: 32)

Dataset	Source	Train %	Validation %
ESC–50	All official folds merged, random split	**80%**	**10%**
sireNNet	All dataset folders concatenated, random split	**80%**	**10%**
LSSiren	Full dataset, random split	**80%**	**10%**
US8K	All folds (1–10) merged, random split	**80%**	**10%**
FSD50K	Development set, random split	**90%**	**10%**

Table 9 Bi-directional cross-dataset evaluation: models trained on external corpora and tested on *AudioSet-EV*

Training dataset	Accuracy	Precision (PPV)	Recall (TPR)	F_1 score	F_2 score	Specificity (TNR)	AuROC	AuPRC
SireNNet	55.56%	55.65%	99.91%	71.41%	88.31%	0.76%	50.33%	50.06%
LSSiren	53.22%	53.22%	100.00%	69.48%	86.99%	0.00%	50.00%	50.00%
ESC-50	48.86%	0.00%	0.00%	0.00%	0.00%	100.00%	50.00%	51.14%
US8K	56.20%	91.87%	15.74%	26.87%	31.81%	98.54%	57.14%	57.55%
FSD50K	52.47%	42.15%	38.92%	40.46%	39.50%	65.87%	51.36%	49.78%

always-negative classifier). The ESC-50 case shows the opposite behavior: perfect specificity but null sensitivity, reflecting a complete failure to generalize beyond its constrained representations. UrbanSound8K achieves slightly better balance, with very high precision (91.9%) but extremely low recall (15.7%), confirming a tendency to overfit to narrow acoustic contexts. The accuracy across datasets ranges between 48% and 56%. Overall, these results validate the central motivation behind `AudioSet-EV`: robust and transferable siren representations emerge from datasets that are both *semantically controlled* and sufficiently *diverse* in their *acoustic content*. Task-specific corpora such as *SireNNet* and *LSSiren* tend to overfit to narrow or homogeneous acoustic contexts, whereas the structural balance and broader variability of `AudioSet-EV` enable improved generalization across real-world urban soundscapes.

Taken together, the uni- and bi-directional evaluations confirm that `AudioSet-EV` provides a stronger and state-of-the-art foundation for training transferable models compared to existing corpora. While potential overlaps between datasets cannot be fully excluded, the consistent performance gaps observed across benchmarks indicate that `AudioSet-EV` captures a broader spectrum of acoustic variability, thereby ensuring both intra-dataset reliability and cross-dataset generalization — a property not achievable with pre-existing corpora alone. Moreover, in a separate work we show how our E-PANNs-derived model emerges as a scalable and interpretable solution for the EV task, extending its applicability to SED protocols [31].

5 Conclusions

This work introduced `AudioSet-Tools` (GitHub), a modular and taxonomy-consistent Python framework designed to overcome the structural and semantic limitations of working with Google's AudioSet. By enabling reproducible filtering, label validation, class rebalancing, and automated audio retrieval, the framework facilitates transparent and task-driven design of semantically consistent audio sub-corpora. In doing so, it addresses key limitations in scalability, label management, and dataset reproducibility, providing a flexible infrastructure for large-scale and real-world machine listening tasks.

To showcase the framework's utility, we presented `AudioSet-EV` (Zenodo), a curated dataset for Emergency Vehicle (EV) siren recognition. Through a systematic design process — enforcing Standardized Audio events Label Taxonomy (SALT) consistency, controlling class imbalance, and applying fine-grained label filtering — we built a corpus that bridges real-world acoustic traffic scenarios and standardized evaluation protocols. Extensive benchmarking with E-PANNs confirmed the dataset's ability to support robust and transferable classifiers across diverse taxonomies and acoustic conditions. In addition, support to the `AudioSet-Strong` release provides segment-level supervision, enabling downstream sound event detection research. Benchmarking of `AudioSet-EV-Strong` is ongoing and will be systematically compared with other available temporally annotated corpora.

A promising research direction involves the adoption of taxonomy-driven inference models, such as the One-Class-One-Network (OCON) architecture [59, 60], which enables modular classification schemes tailored to individual taxonomic nodes or leaves. These strategies could exploit the structural consistency of AudioSet-derived graphs and complement the flexibility of E-PANNs (or similar)-based pipelines, supporting scalable deployment across tasks of increasing granularity and semantic complexity.

Several limitations emerged during development and experimentation. First, the current `AudioSet-Tools StandardDownloader`, though functional, lacks multiprocessing and adaptive bandwidth strategies, which constrain performance in high-throughput environments. Planned improvements include dynamic resource allocation and browser-based fallback mechanisms, currently tested only under Linux systems.

Second, while the `AudioSet-EV` *Negatives* subset is numerically balanced, semantic or perceptual balancing is not enforced: some samples are easily distinguishable from sirens, whereas others (e.g., alarms, horns) exhibit high acoustic similarity, potentially skewing training dynamics. Furthermore, the entire corpus remains

tightly coupled to the availability of YouTube-hosted content, introducing long-term reproducibility risks. To address these issues, we envision the development of an update layer that incorporates machine-readable exception lists for unavailable segments and fosters community-driven initiatives to extend, restructure, or reconstruct AudioSet itself.

Future extensions of our framework could also include active data collection modules, enabling users to query and retrieve new candidate material for specific classes directly from online repositories, as well as annotation support interfaces capable of handling both weak (tags) and strong (time-aligned) labels. These functionalities would facilitate the integration of newly collected or user-provided audio while ensuring compliance with the AudioSet taxonomy.

From our perspective, `AudioSet-Tools` represents the enabling core of a broader service vision, currently under development. This service provides a browser-based interface (`HTML`/`JavaScript`) for the collection, management, and maintenance of datasets for audio tagging, acoustic scene classification, and sound event detection, with support for well-known sources (e.g., `Soundata` [61], Zenodo). Leveraging a modular, tab-based design with user session support, it covers the entire data workflow, from automated download and metadata pre-processing to learning-driven *semantic re-mapping* via SALT. A dedicated audio/metadata interface enables sample-level inspection, label editing, and insertion of new tags or event tracks, while also supporting the exchange of structured session files. Additional features under development include the ability to restructure or extend original annotations, integrate event-level labels where missing, and share annotator confidence maps to facilitate community-driven refinement. Together, these functionalities aim to foster long-term maintainability of AudioSet-compliant datasets and to lower barriers for audio collaborative curation.

Regarding data augmentation, the current design includes only simple, reproducible time-domain transformations. These do not account for realistic environmental propagation phenomena (e.g., reverberation, occlusion, multi-path effects) and are not yet conditioned on class- or context-specific variables. Future work will address these gaps by incorporating context-aware and physically informed augmentations.

From a integration perspective, we plan to embed real-time retrieval capabilities within PyTorch `Dataset` and `DataLoader` classes, allowing batch-wise on-the-fly audio loading to reduce disk I/O and streamline training workflows. Planned extensions also include support for additional SALT-compliant datasets such as VGGSound [62], MAESTRO-Real [63], Urbansas [64], and SONYC [65, 66], further expanding the EV-benchmark ecosystem.

In summary, `AudioSet-Tools` and `AudioSet-EV` offer a flexible and reproducible foundation for constructing task-specific audio datasets. By openly releasing our tools and datasets, we aim to catalyze a new standard for transparency and efficiency in large-scale audio curation — lowering entry barriers, fostering cross-dataset comparability, and paving the way for more inclusive and deployment-ready sound recognition systems.

Abbreviations

API	Application Programming Interface
ASC	Acoustic Scene Classification
AT	Audio Tagging
AudioSet-EV	AudioSet Emergency Vehicle (siren recognition) subset
AuPRC	Area under the Precision–Recall Curve
AuROC	Area under the Receiver Operating characteristic Curve
CNN	Convolutional Neural Network
CNIT	National Inter-University Consortium for Telecommunications
CSV	Comma-Separated Values
DASH	Dynamic Adaptive Streaming over HTTP
DISIM	Department of Information Engineering, Computer Science and Mathematics (UnivAQ)
DL	Deep Learning
E-PANNs	Efficient Pre-trained Audio Neural Networks
ESC-50	Environmental Sound Classification 50
EV	Emergency Vehicle(s)
F1, F2	*F*-scores (beta = 1, 2)
FSD50K	FreeSound Dataset 50K
GP-AT	General-Purpose Audio Tagging
GPU	Graphics Processing Unit
HLS	HTTP Live Streaming
HRF	Human-Readable Format
HTML	HyperText Markup Language
ID	Identifier
LSSiren	Large Scale Siren dataset
MAESTRO Real	Multi-Annotator Estimated Strong Labels (subset)
MCC	Matthews Correlation Coefficient
MID	Machine ID
NN	Neural Network(s)
OCON	One-Class-One-Network
PANNs	Pre-trained Audio Neural Networks
PNRR	Piano Nazionale di Ripresa e Resilienza
PPV	Positive Predictive Value (precision)
PR	Precision–Recall
ROC	Receiver Operating Characteristic
SALT	Standardized Audio events Label Taxonomy
SED	Sound Event Detection
SONYC-UST	Sounds of New York City – Urban Sound Tagging
SSL	Sound Source Localization
SVD	Singular Value Decomposition
TN	True Negative
TNR	True Negative Rate (Specificity)
TP	True Positive
TPR	True Positive Rate (Recall)
TSV	Tab-Separated Values
US8K	UrbanSound8K
UnivAQ	University of L'Aquila

Acknowledgements
Not applicable.

Authors' contributions
SG is the author of the AudioSet-Tools framework and designed the core mechanisms for structuring, analyzing, and re-distributing the AudioSet-EV dataset and related research assets. He was also the major contributor to the

writing of the manuscript, with the valuable and kind support of CR and MG, who actively contributed through regular discussions and technical feedback. FG supervised the overall progress of the work and the development of the manuscript. All authors read and approved the final manuscript.

Funding
This work was partially supported by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, partnership on "Telecommunications of the Future" (PE00000001 - program "RESTART") and by the European Union - NextGenerationEU under the Italian Ministry of University and Research (MUR) National Innovation Ecosystem grant ECS00000041 - VITALITY - CUP E13C22001060006.

Data Availability
The datasets generated during the current study are available in the Zenodo repository, [https://zenodo.org/records/14882314]; related analysis assets are available on GitHub repositories: [https://github.com/StefanoGiacomelli/audioset-tools/tree/main] and [https://github.com/StefanoGiacomelli/e2panns].
AudioSet-Tools
• Project name: AudioSet-Tools
• Project home page: https://github.com/StefanoGiacomelli/audioset-tools/
• Archived version: DOI - 10.5281/zenodo.14882314
• Operating system(s): Platform independent (Linux tested)
• Programming language: Python
• Other requirements: FFMpeg 7.1.1, yt-dlp 2025.05.22, Python requirements provided in repository file
• License: GNU GPLv3
• Any restrictions to use by non-academics: license needed.

Declarations

Competing interests
Not applicable.

Received: 5 June 2025 Accepted: 3 November 2025
Published online: 02 December 2025

References

1. J. Abeßer, A review of deep learning based methods for acoustic scene classification. Appl. Sci. (2020). https://doi.org/10.3390/app10062020
2. B. Ding et al., Acoustic scene classification: a comprehensive survey. Expert Syst. Appl. **238**, 121902 (2024). https://doi.org/10.1016/j.eswa.2023.121902
3. S. Chandrakala, S.L. Jayalakshmi, Environmental audio scene and sound event recognition for autonomous surveillance: a survey and comparative studies. ACM Comput. Surv. **52**(3) (2019). https://doi.org/10.1145/3322240
4. A. Mesaros, T. Heittola, T. Virtanen, M.D. Plumbley, Sound event detection: a tutorial. IEEE Signal Process. Mag. **38**(5), 67–83 (2021). https://doi.org/10.1109/MSP.2021.3090678
5. C. Castorena, M. Cobos, J. Lopez-Ballester, F.J. Ferri, A safety-oriented framework for sound event detection in driving scenarios. Appl. Acoust. **215**, 109719 (2024). https://doi.org/10.1016/j.apacoust.2023.109719
6. Z. Mnasri, S. Rovetta, F. Masulli, Anomalous sound event detection: a survey of machine learning based methods and applications. Multimed. Tools Appl. **81**(4), 5537–5586 (2022). https://doi.org/10.1007/s11042-021-11817-9
7. P.A. Grumiaux, S. Kitić, L. Girin, A. Guérin, A survey of sound source localization with deep learning methods. J. Acoust. Soc. Am. **152**(1), 107–151 (2022). https://doi.org/10.1121/10.0011809
8. J.F. Gemmeke, D.P.W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R.C. Moore, M. Plakal, M. Ritter, in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Audio Set: An ontology and human-labeled dataset for audio events (IEEE, New Orleans, 2017), pp. 776–780. https://doi.org/10.1109/ICASSP.2017.7952261
9. S. Roller, D. Kiela, M. Nickel. Hearst Patterns Revisited: Automatic Hypernym Detection from Large Text Corpora (2018). https://doi.org/10.48550/arXiv.1806.03191
10. T. Pellissier Tanon, D. Vrandečić, S. Schaffert, T. Steiner, L. Pintscher, in *Proceedings of the 25th International Conference on World Wide Web*, From Freebase to Wikidata: The Great Migration, WWW '16 (International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 2016), pp. 1419–1428. https://doi.org/10.1145/2872427.2874809
11. N. Turpault, R. Serizel, E. Vincent, in *2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2020)*, Limitations of Weak Labels for Embedding and Tagging (2020), pp. 131–135. https://doi.org/10.1109/ICASSP40776.2020.9053160
12. S. Damiano, T. Dietzen, T.v. Waterschoot. Frequency Tracking Features for Data-Efficient Deep Siren Identification (2024). https://doi.org/10.48550/arXiv.2409.08587
13. V.T. Tran, W.H. Tsai, Acoustic-based emergency vehicle detection using convolutional neural networks. IEEE Access **8**, 75702–75713 (2020). https://doi.org/10.1109/ACCESS.2020.2988986
14. A. Shah, A. Singh, A. Singh, Audio Classification of Emergency Vehicle Sirens Using Recurrent Neural Network Architectures, in *Proceedings of International Conference on Paradigms of Communication, Computing and Data Analytics*, ed. by A. Yadav, S.J. Nanda, M.H. Lim (Springer Nature, Singapore, 2023), pp.71–83. https://doi.org/10.1007/978-981-99-4626-6_6
15. S. Hershey, D.P.W. Ellis, E. Fonseca, A. Jansen, C. Liu, R. Channing Moore, M. Plakal, in *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, The Benefit of Temporally-Strong Labels in Audio Event Classification (2021), pp. 366–370.https://doi.org/10.1109/ICASSP39728.2021.9414579
16. B. Ahmed. audioset-utils GitHub (2024). https://github.com/bilalhsp/audioset-utils. Accessed 3 Nov 2025
17. A. McDonagh. audioset-processing GitHub (2025). https://github.com/aoifemcdonagh/audioset-processing. Accessed 3 Nov 2025
18. K. Lee. Fast-Audioset-Download GitHub (2025). https://github.com/dlrudco/Fast-Audioset-Download. Accessed 3 Nov 2025
19. yt-dlp GitHub (2025). https://github.com/yt-dlp/yt-dlp. Accessed 3 Nov 2025
20. S. Tomar, Converting video formats with FFmpeg. Linux J. **2006**(146), 10 (2006)
21. J. Salamon, C. Jacoby, J.P. Bello, in *Proceedings of the 22nd ACM international conference on Multimedia*, A Dataset and Taxonomy for Urban Sound Research (ACM, Orlando Florida USA, 2014), pp. 1041–1044. https://doi.org/10.1145/2647868.2655045
22. E. Fonseca, X. Favory, J. Pons, F. Font, X. Serra. FSD50K: An Open Dataset of Human-Labeled Sound Events (2022). https://doi.org/10.48550/arXiv.2010.00475
23. P. Stamatiadis, M. Olvera, S. Essid. SALT: Standardized Audio event Label Taxonomy (2024). https://doi.org/10.48550/arXiv.2409.11746
24. H. Liu et al., in *Interspeech 2023*, Ontology-aware Learning and Evaluation for Audio Tagging (2023), pp. 3799–3803. https://doi.org/10.21437/Interspeech.2023-979
25. J. Liang, H. Phan, E. Benetos, in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Learning from Taxonomy: Multi-Label Few-Shot Classification for Everyday Sound Recognition (2024), pp. 771–775. https://doi.org/10.1109/ICASSP48485.2024.10446908
26. py-salt GitHub. https://github.com/tpt-adasp/salt/tree/main/py-salt. Accessed 3 Nov 2025
27. A. Shah, A. Singh. sireNNet-Emergency Vehicle Siren Classification Dataset For Urban Applications (2023). https://doi.org/10.17632/j4ydzzv4kb.1
28. M. Usaid, M. Asif, t. rajab, P.D.E.S. Hussain, P.D.s.M. munaf, S. Wasi. Large-Scale Audio Dataset for Emergency Vehicle Sirens and Road Noises (2022). https://doi.org/10.6084/M9.FIGSHARE.19291472.V2
29. M. Asif, M. Usaid, M. Rashid, T. Rajab, S. Hussain, S. Wasi, Large-scale audio dataset for emergency vehicle sirens and road noises. Sci. Data **9**(1), 599 (2022). https://doi.org/10.1038/s41597-022-01727-2
30. K.J. Piczak, in *Proceedings of the 23rd ACM international conference on Multimedia*, ESC: Dataset for Environmental Sound Classification (Association for Computing Machinery, New York, 2015), MM '15, pp. 1015–1018. https://doi.org/10.1145/2733373.2806390
31. S. Giacomelli et al., From Large-scale Audio Tagging to Real-Time Explainable Emergency Vehicle Sirens Detection (2025). https://doi.org/10.48550/arXiv.2506.23437

32. M.L. Quatra. audioset-download GitHub (2025). https://github.com/MorenoLaQuatra/audioset-download. Accessed 3 Nov 2025
33. C.R. Harris, K.J. Millman, S.J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N.J. Smith, R. Kern, M. Picus, S. Hoyer, M.H. Van Kerkwijk, M. Brett, A. Haldane, J.F. Del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T.E. Oliphant, Array programming with NumPy. Nature **585**(7825), 357–362 (2020). https://doi.org/10.1038/s41586-020-2649-2
34. J.O. Smith, *Digital Audio Resampling Home Page* (Center for Computer Research in Music and Acoustics, 2002). https://ccrma.stanford.edu/~jos/resample/. Accessed 3 Nov 2025
35. Y.Y. Yang, M. Hira, Z. Ni, A. Chourdia, A. Astafurov, C. Chen, C.F. Yeh, C. Puhrsch, D. Pollack, D. Genzel, D. Greenberg, E.Z. Yang, J. Lian, J. Mahadeokar, J. Hwang, J. Chen, P. Goldsborough, P. Roy, S. Narenthiran, S. Watanabe, S. Chintala, V. Quenneville-Bélair, Y. Shi. TorchAudio: Building Blocks for Audio and Speech Processing (2022). https://doi.org/10.48550/arXiv.2110.15018
36. J. Hwang, M. Hira, C. Chen, X. Zhang, Z. Ni, G. Sun, P. Ma, R. Huang, V. Pratap, Y. Zhang, A. Kumar, C.Y. Yu, C. Zhu, C. Liu, J. Kahn, M. Ravanelli, P. Sun, S. Watanabe, Y. Shi, Y. Tao, R. Scheibler, S. Cornell, S. Kim, S. Petridis. TorchAudio 2.1: Advancing speech recognition, self-supervised learning, and audio processing components for PyTorch (2023). https://doi.org/10.48550/arXiv.2310.17864
37. B. McFee, A. Valentino, S. Kruchinin, E. Karpov, N. Werner, W. Pimenta, chris, endolith. bmcfee/resampy: 0.4.3 (2024). https://doi.org/10.5281/ZENODO.596633
38. K. Choudhury, D. Nandi, Review of emergency vehicle detection techniques by acoustic signals. Trans. Indian Natl. Acad. Eng. **8**(4), 535–550 (2023). https://doi.org/10.1007/s41403-023-00424-9
39. C. Castorena, M. Cobos, J. Lopez-Ballester, F.J. Ferri, A safety-oriented framework for sound event detection in driving scenarios. Appl. Acoust. **215**, 109719 (2024). https://doi.org/10.1016/j.apacoust.2023.109719
40. D&R Electronics. An overview of emergency vehicle sirens. (2021). https://www.dandrelectronics.com/an-overview-of-emergency-vehicle-sirens.html. Accessed 3 Nov 2025
41. LED Equipped. Different siren sounds used by first responders. (2022). https://www.ledequipped.com/blogs/news/different-siren-sounds-used-by-first-responders. Accessed 3 Nov 2025
42. BossHorn. Different fire truck siren sounds: An overview. (2021). https://bosshorn.com/blogs/blog/different-fire-truck-siren-sounds. Accessed 3 Nov 2025
43. Extreme Tactical Dynamics. What are the different sounds a police siren makes. (2020). https://www.extremetacticaldynamics.com/blog/what-are-the-different-sounds-a-police-siren-makes/. Accessed 3 Nov 2025
44. A.E. Ramirez, E. Donati, C. Chousidis, A siren identification system using deep learning to aid hearing-impaired people. Eng. Appl. Artif. Intell. **114**, 105000 (2022). https://doi.org/10.1016/j.engappai.2022.105000
45. M.Y. Shams, T. Abd El-Hafeez, E. Hassan, Acoustic data detection in large-scale emergency vehicle sirens and road noise dataset. Expert Syst. Appl. **249**, 123608 (2024). https://doi.org/10.1016/j.eswa.2024.123608
46. U. Mittal, P. Chawla, Acoustic based emergency vehicle detection using ensemble of deep learning models. Procedia Comput. Sci. **218**, 227–234 (2023). https://doi.org/10.1016/j.procs.2023.01.005
47. M. Cantarini, L. Gabrielli, S. Squartini, Few-shot emergency siren detection. Sensors **22**(12), 4338 (2022). https://doi.org/10.3390/s22124338
48. L. Marchegiani, P. Newman, Listening for sirens: locating and classifying acoustic alarms in city scenes. IEEE Trans. Intell. Transp. Syst. **23**(10), 17087–17096 (2022). https://doi.org/10.1109/TITS.2022.3158076
49. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library (2019). https://doi.org/10.48550/arXiv.1912.01703
50. W. Falcon, J. Borovec, A. Wälchli, N. Eggert, J. Schock, J. Jordan, N. Skafte, Ir1dXD, V. Bereznyuk, E. Harris, T. Murrell, P. Yu, S. Præsius, T. Addair, J. Zhong, D. Lipin, S. Uchida, S. Bapat, H. Schröter, B. Dayma, A. Karnachev, A. Kulkarni, S. Komatsu, Martin.B, J.B. Schiratti, H. Mary, D. Byrne, C. Eyzaguirre, cinjon, A. Bakhtin. PyTorchLightning/pytorch-lightning: 0.7.6 release (2020). https://doi.org/10.5281/zenodo.3828935
51. D.S. Park, W. Chan, Y. Zhang, C.C. Chiu, B. Zoph, E.D. Cubuk, Q.V. Le, in *Interspeech 2019*, SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition (ISCA, 2019), pp. 2613–2617. https://doi.org/10.21437/Interspeech.2019-2680
52. H. Zhang, M. Cisse, Y.N. Dauphin, D. Lopez-Paz. mixup: Beyond Empirical Risk Minimization (2018). https://doi.org/10.48550/arXiv.1710.09412
53. P.Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, C. Feichtenhofer. Masked Autoencoders that Listen (2023). https://doi.org/10.48550/arXiv.2207.06405
54. K. Koutini, J. Schlüter, H. Eghbal-zadeh, G. Widmer, in *Interspeech 2022*, Efficient Training of Audio Transformers with Patchout (ISCA, 2022), pp. 2753–2757. https://doi.org/10.21437/Interspeech.2022-227
55. A. Singh, H. Liu, M.D. Plumbley. E-PANNs: Sound Recognition Using Efficient Pre-trained Audio Neural Networks (2023). https://doi.org/10.48550/arXiv.2305.18665
56. Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, M.D. Plumbley, PANNs: large-scale pretrained audio neural networks for audio pattern recognition. IEEE/ACM Trans. Audio Speech Lang. Process. **28**, 2880–2894 (2020). https://doi.org/10.1109/TASLP.2020.3030497
57. A. Singh, P. Rajan, A. Bhavsar, SVD-based redundancy removal in 1-D CNNs for acoustic scene classification. Pattern Recogn. Lett. **131**, 383–389 (2020). https://doi.org/10.1016/j.patrec.2020.02.004
58. A. Singh, M.D. Plumbley, in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Efficient Similarity-Based Passive Filter Pruning for Compressing CNNS (2023), pp. 1–5. https://doi.org/10.1109/ICASSP49357.2023.10095560. ISSN: 2379-190X
59. S. Giacomelli, M. Giordano, C. Rinaldi, in *2024 IEEE Symposium on Computers and Communications (ISCC)*, The OCON model: an old but gold solution for distributable supervised classification (2024), pp. 1–7. https://doi.org/10.1109/ISCC61673.2024.10733621
60. S. Giacomelli, M. Giordano, C. Rinaldi, in *2024 IEEE 5th International Symposium on the Internet of Sounds (IS2)*, The OCON Model: An Old but Green Solution for Distributable Supervised Classification for Acoustic Monitoring in Smart. Cities (2024), pp. 1–10. https://doi.org/10.1109/IS262782.2024.10704155
61. M. Fuentes et al., Soundata: reproducible use of audio datasets. J. Open Source Softw. **9**(98), 6634 (2024). https://doi.org/10.21105/joss.06634
62. H. Chen, W. Xie, A. Vedaldi, A. Zisserman, in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Vggsound: A Large-Scale Audio-Visual Dataset (2020), pp. 721–725. https://doi.org/10.1109/ICASSP40776.2020.9053174
63. I.M. Morato, M. Harju, A. Mesaros, MAESTRO Real - Multi-Annotator Estimated Strong Labels (2023). https://doi.org/10.5281/zenodo.7244360
64. M. Fuentes, B. Steers, P. Zinemanas, M. Rocamora, L. Bondi, J. Wilkins, Q. Shi, Y. Hou, S. Das, X. Serra, J.P. Bello, in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Urban Sound & Sight: Dataset And Benchmark For Audio-Visual Urban Scene Understanding (2022), pp. 141–145. https://doi.org/10.1109/ICASSP43922.2022.9747644
65. M. Cartwright, A.E.M. Mendez, A. Cramer, V. Lostanlen, G. Dove, H.H. Wu, J. Salamon, O. Nov, J. Bello. SONYC Urban Sound Tagging (SONYC-UST): A Multilabel Dataset from an Urban Acoustic Sensor Network (2019). https://doi.org/10.33682/j5zw-2t88
66. M. Cartwright, J. Cramer, A.E.M. Mendez, Y. Wang, H.H. Wu, V. Lostanlen, M. Fuentes, G. Dove, C. Mydlarz, J. Salamon, O. Nov, J.P. Bello. SONYC-UST-V2: an urban sound tagging dataset with spatiotemporal context (2020). https://doi.org/10.48550/arXiv.2009.05188

Publisher's Note